

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Development of an Online Programming Teaching and Management System

NGUYỄN THẾ ĐẠT

dat.nt213605@sis.hust.edu.vn

Program: Electronics and Telecommunications Engineering

Supervisor: Doctor of Philosophy Nguyễn Quang Minh

Department: Electronics and Telecommunications Engineering

School: School of Electrical and Electronic Engineering

HANOI, 01/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Development of an Online Programming Teaching and Management System

NGUYỄN THẾ ĐẠT
dat.nt213605@sis.hust.edu.vn

Program: Electronics and Telecommunications Engineering

Supervisor: Doctor of Philosophy Nguyễn Quang Minh _____

Signature

Department: Electronics and Telecommunications Engineering

School: School of Electrical and Electronic Engineering

HANOI, 01/2026

ACKNOWLEDGMENTS

The students' sincere thanks should send to their lover, family, friends, teachers, and themselves for their hard work and determination to achieve the best results. The acknowledgments should be written in a concrete manner and limited in the range of 100-150 words.

ABSTRACT

In the context of rapid digital transformation, the demand for learning programming has increased significantly as programming becomes an essential skill for learners and the labor market. However, many learners still face difficulties due to unclear learning pathways, a lack of continuous hands-on practice environments, and the absence of detailed assessment and progress-tracking mechanisms for each lesson. Currently, common approaches include: online learning platforms that mainly provide videos and learning materials; learning management systems (LMS) that focus on class and content administration; and online judge (OJ) systems that concentrate on programming assignment grading. Nevertheless, these approaches are often fragmented between theory and practice, provide an unsmooth learning experience, and may introduce security risks when executing untrusted source code without proper isolation. To address these issues, this project adopts an integrated web-based programming learning platform approach that connects the entire learning process from “learning–practice–assessment–tracking.” The system is implemented using a client–server architecture with ReactJS for the user interface, Spring Boot for RESTful APIs and business logic, and MySQL for centralized data storage. The platform supports learning through videos, materials, and quizzes; enables coding practice directly on a web-based editor with automatic grading via Judge0 combined with a Docker sandbox to ensure safe code execution; integrates a virtual assistant for debugging support based on Google Gemini; applies role-based access control (RBAC) for learners, instructors, teaching assistants, and administrators; and integrates VNPay payments with digital signatures and automatic course activation. The final system operates stably, provides detailed progress tracking, supports efficient content administration, and enhances the overall experience of online programming learning.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and scope of the graduation thesis	2
1.3 Tentative solution	2
1.4 Thesis organization.....	4
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	5
2.1 Status survey	5
2.1.1 Survey of Similar Products.....	5
2.1.2 User Needs Survey	6
2.2 Functional Overview	7
2.2.1 General use case diagram.....	7
2.2.2 Detailed use case diagram.....	9
2.2.3 Business process	14
2.3 Functional description.....	15
2.3.1 Use Case Specification: Login	15
2.3.2 Use Case Specification: Enroll and Pay for Courses	16
2.3.3 Use Case Specification: Take Coding Exercises	17
2.3.4 Use Case Specification: Manage Courses	18
2.3.5 Use Case Specification: Approve Courses	19
2.3.6 Use Case Specification: Online Interaction and Discussion.....	20
2.3.7 Use Case Specification: User Management.....	21
2.4 Non-functional requirement.....	22
2.4.1 Performance	22
2.4.2 Reliability	22

2.4.3 Usability	23
2.4.4 Security	23
2.4.5 Maintainability	24
2.4.6 Scalability	24
CHAPTER 3. METHODOLOGY.....	25
3.1 Technologies and Platforms	25
3.1.1 Java Programming Language	25
3.1.2 Spring Boot Framework.....	26
3.1.3 ReactJS	28
3.1.4 MySQL Database.....	30
3.1.5 Other Technologies	30
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	32
4.1 Architecture design.....	32
4.1.1 Software architecture selection	32
4.1.2 Overall design.....	34
4.1.3 Detailed package design	36
4.2 Detailed design.....	37
4.2.1 User interface design	37
4.2.2 Layer design	42
4.2.3 Database design	51
4.3 Application Building.....	62
4.3.1 Libraries and Tools.....	62
4.3.2 Achievement.....	64
4.3.3 Illustration of main functions	65
4.4 Testing.....	71
4.4.1 Testing the Register / Login Function.....	72

4.4.2 Testing the Pay for Courses Function	74
4.4.3 Testing the Take Coding Exercises Function	75
4.5 Deployment	77
CHAPTER 5. SOLUTION AND CONTRIBUTION	79
5.1 Secure and High-Performance Automatic Code Evaluation Architecture....	79
5.1.1 Problem Statement	79
5.1.2 Proposed Solution	79
5.1.3 Achieved Results.....	80
5.2 Programming Virtual Assistant Integrated with Google Gemini AI	80
5.2.1 Problem Statement	80
5.2.2 Proposed Solution	80
5.2.3 Achieved Results.....	81
5.3 Teaching Assistant (TA) Management System and Advanced Authoriza- tion	81
5.3.1 Problem Statement	81
5.3.2 Proposed Solution	81
5.3.3 Achieved Results.....	82
5.4 Dynamic Statistics Computation Mechanism and Data Consistency	82
5.4.1 Problem Statement	82
5.4.2 Proposed Solution	82
5.4.3 Achieved Results.....	83
5.5 Secure Payment Integration with VNPay and Digital Signature.....	83
5.5.1 Problem Statement	83
5.5.2 Proposed Solution	83
5.5.3 Achieved Results.....	84

5.6 Standard RESTful API Architecture and Multi-layer Security	84
5.6.1 Problem Statement	84
5.6.2 Proposed Solution	84
5.6.3 Achieved Results.....	85
5.7 Summary	85
CHAPTER 6. CONCLUSION AND FUTURE WORK	86
6.1 Conclusion.....	86
6.2 Future Work.....	87
REFERENCE	90

LIST OF FIGURES

Figure 2.1	General Use Case	8
Figure 2.2	Detailed Search Courses Diagram	9
Figure 2.3	Detailed Enroll and Pay for Courses Diagram	10
Figure 2.4	Detailed Manage Personal Information Diagram	10
Figure 2.5	Detailed Online Interaction and Discussion Diagram	11
Figure 2.6	Detailed Participate in Online Learning Diagram	11
Figure 2.7	Detailed Manage Courses Diagram	12
Figure 2.8	Detailed Manage Exams and Assignments Diagram	12
Figure 2.9	Detailed Manage and Support Learners Diagram	13
Figure 2.10	Detailed User Management Diagram	13
Figure 2.11	Detailed Operations and Marketing Management Diagram	14
Figure 2.12	Learning and Programming Practice Process	15
Figure 2.13	Course Content Development and Approval Process	23
Figure 3.1	Spring Boot Layered Architecture	27
Figure 3.2	ReactJS Virtual DOM Architecture	29
Figure 4.1	Client–Server Architecture Model (RESTful API)	32
Figure 4.2	System Overview Diagram	34
Figure 4.3	Ví dụ thiết kế gói	36
Figure 4.4	Homepage UI Wireframe	39
Figure 4.5	Course Detail Page UI Wireframe	40
Figure 4.6	Lesson Display UI Wireframe	41
Figure 4.7	Course Enrollment and Payment Sequence Diagram	49
Figure 4.8	Course Creation and Approval Sequence Diagram	50
Figure 4.9	Lesson Learning and Progress Update Sequence Diagram	51
Figure 4.10	Entity–Relationship Diagram (ERD)	52
Figure 4.11	Home Page Interface	65
Figure 4.12	Interface displaying featured courses	66
Figure 4.13	The Course Detail Interface	66
Figure 4.14	Entity–Relationship Diagram (ERD)	67
Figure 4.15	The Learning Material Interface	67
Figure 4.16	The Quiz Interface	68
Figure 4.17	The Coding Practice Interface	68
Figure 4.18	The Course Management Interface	69
Figure 4.19	The Admin Dashboard Interface	69

Figure 4.20 The Learner Progress Tracking Interface	70
Figure 4.21 The Detailed Learner Progress Interface	71

LIST OF TABLES

Table 2.1	Comparison of Similar Online Programming Learning Platforms	6
Table 4.1	Screen Information Specification	37
Table 4.2	Button Design Standardization	37
Table 4.3	Control Design Standardization	38
Table 4.4	Feedback Message Rules	38
Table 4.5	UI Color Palette	39
Table 4.6	User Class	42
Table 4.7	Course Class	44
Table 4.8	Learning Class	45
Table 4.9	Payment Class	46
Table 4.10	CourseService Class	47
Table 4.11	LearningService Class	48
Table 4.12	users Table Design	52
Table 4.13	categories Table Design	53
Table 4.14	course Table Design	53
Table 4.15	course_modules Table Design	53
Table 4.16	lesson Table Design	54
Table 4.17	learning Table Design	54
Table 4.18	progress Table Design	54
Table 4.19	lesson_progress Table Design	55
Table 4.20	comments Table Design	55
Table 4.21	direct_questions Table Design	56
Table 4.22	ta_course_assignments Table Design	56
Table 4.23	ta_reminders Table Design	56
Table 4.24	payments Table Design	57
Table 4.25	payment_orders Table Design	57
Table 4.26	payment_courses Table Design	58
Table 4.27	carts Table Design	58
Table 4.28	news Table Design	58
Table 4.29	banners Table Design	59
Table 4.30	messages Table Design	59
Table 4.31	notifications Table Design	59
Table 4.32	exams Table Design	60

Table 4.33	exam_questions Table Design	60
Table 4.34	exam_submissions Table Design	60
Table 4.35	exam_submission_answers Table Design	61
Table 4.36	code_exercises Table Design	61
Table 4.37	assessment Table Design	62
Table 4.38	ai_test_history Table Design	62
Table 4.39	List of Libraries and Tools Used (Summarized)	63
Table 4.40	Detailed Application Information Statistics	64
Table 4.41	Register / Login Function Testing	72
Table 4.42	Pay for Courses Function Testing	74
Table 4.43	Table 4.5: Take Coding Exercises Function Testing	75
Table 4.44	Backend and Frontend (Web) Server Information	78
Table 4.45	Database Server Information	78

LIST OF ABBREVIATIONS

Abriviation	Full Expression
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHAPTER 1. INTRODUCTION

1.1 Motivation

At present, along with the rapid development of information technology and the process of digital transformation in education, the demand for learning and improving programming skills is increasingly growing. Programming has become an essential skill, not only for students in information technology-related majors but also for many other groups such as working professionals, self-learners, and individuals seeking career transitions. As a result, online programming education has been increasingly adopted due to its flexibility and the absence of constraints related to space and time.

In practice, current online programming teaching activities have met several basic learning needs, including the provision of theoretical lectures, learning materials, and instructional content. Learners can access knowledge in a relatively convenient and proactive manner compared to traditional training methods. However, for a field with a strong emphasis on practice such as programming, these functionalities remain insufficient to ensure comprehensive learning effectiveness.

One of the most significant limitations lies in the inadequate and unstructured support for programming practice. Programming requires learners to frequently write code, test results, and debug during the learning process. When these activities are not closely integrated with instructional content, learners may encounter difficulties in applying acquired knowledge to specific problems. This situation reduces knowledge absorption and directly affects learning quality.

In addition, course management and the monitoring of learners' learning progress in online environments still face many challenges. As the number of learners and instructional content continues to increase, tracking individual learning progress, lesson completion levels, and learning outcomes becomes increasingly complex. If such information is not managed in a complete and accurate manner, instructors and training institutions will encounter difficulties in evaluating training quality and adjusting instructional content appropriately.

Furthermore, the level of interaction between learners and instructors in online programming courses remains limited. When learners encounter errors during practice or fail to fully understand certain concepts, the lack of timely support may disrupt the learning process. In programming courses, where accurate understanding and correct practice are crucial, this limitation significantly affects learners'

experiences and learning motivation.

If the aforementioned issues are effectively addressed, numerous important benefits can be achieved. First, the quality of programming teaching and learning in online environments will be enhanced, enabling learners to access knowledge and develop skills more effectively. At the same time, course management, instructional content, and learning processes will become clearer and more consistent, supporting instructors and training institutions in monitoring and evaluation activities. Moreover, the achieved results can be applied to other online training domains with similar characteristics, thereby contributing to the improvement of educational quality in the context of the current digital transformation.

1.2 Objectives and scope of the graduation thesis

Existing online programming learning platforms in Vietnam include CodeLearn, Unica, and F8-Education. In general, these platforms have performed relatively well in providing learning content in the form of courses and learning paths, supporting learners in accessing video lectures and learning materials, and offering certain types of exercises or assessments to reinforce knowledge. However, learners' needs do not stop at merely watching lectures.

In practice, learners still face difficulties when engaging in programming practice closely associated with lessons, completing assessments and receiving feedback, as well as clearly tracking their learning progress. In addition, the management of course content, examinations, and programming assignments for teachers and training institutions remains limited in terms of centralization and scalability.

Therefore, this thesis aims to develop an Online Programming Teaching and Management System with the following main functionalities:

- Providing learning and practice content, including watching video lectures, reading materials, completing quizzes, and performing coding exercises.
- Managing courses, examinations, and programming assignments.
- Managing users according to different roles.
- Learner Management and Support.
- Searching for, registering for, and purchasing courses.

1.3 Tentative solution

(i) In order to address the aforementioned practical issues, this thesis adopts the approach of developing the system as a web-based application following the client-server architecture. In terms of technology, ReactJS is employed to develop the

user interface with the goal of ensuring an intuitive user experience, fast response time, and ease of use across multiple platforms. The backend is implemented using Spring Boot to provide robust RESTful APIs for handling complex business logic. MySQL is selected as the database management system to store data in a centralized manner, ensuring data integrity, consistency, and good scalability as the number of records related to users, assignments, and courses increases. The combination of these technologies enables the system to be easily deployed, maintained, and to effectively meet performance requirements.

(ii) Based on this technological foundation, a comprehensive online programming teaching and management system is developed to meet the needs of four main user groups: learners, instructors, teaching assistants, and administrators. For learners, the system supports course participation through diverse content formats (videos, documents, quizzes), while also providing an environment for practicing coding exercises and taking online tests or examinations to enable continuous assessment of learning outcomes; in addition, it supports operational activities such as searching for, registering for, and purchasing courses. For instructors and administrators, the system provides advanced management tools to support training organization, including course management, learning material development, examination question banks, and user management. For teaching assistants, the system offers specific authorization mechanisms that allow them to support instructors in class management activities, including answering learners' questions in discussion areas and lesson sections, as well as monitoring individual learning progress in order to provide timely support when necessary. Furthermore, the system integrates direct messaging and communication features, creating an effective communication bridge between learners and the teaching team (instructors and teaching assistants), thereby enhancing interaction in the online learning environment.

(iii) The main contribution of this thesis is the successful development of a closed-loop online programming learning platform that integrates the processes of Learning – Practice – Assessment – and Management into a unified system. The system not only facilitates learners' access to knowledge and the development of programming skills, but also optimizes the coordination of teaching activities. In particular, the incorporation of the teaching assistant role into the system helps reduce the workload of primary instructors, accelerates response time, and improves learner support, thereby contributing to the enhancement of overall training quality in online education environments.

1.4 Thesis organization

The remainder of this graduation thesis is organized as follows.

Chapter 2 presents a survey of the current state of existing online programming teaching systems and identifies aspects that can be improved. Based on this survey, the chapter analyzes and designs the system functionalities, specifies the main functional features, and clarifies the non-functional requirements.

Chapter 3 introduces the technologies and platforms selected for system development, including front-end technologies, server-side technologies, and the database system, and describes the roles of these technologies within the overall architecture.

Chapter 4 describes the system design and implementation process, including architectural design, database design, user interface design, and the implementation of the system's core functionalities.

Chapter 5 summarizes the solutions and notable contributions achieved during the development process, and evaluates the implementation results in comparison with the predefined objectives.

Chapter 6 concludes the thesis by summarizing the achieved results, identifying existing limitations, and proposing future development directions to further improve the system.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

This chapter presents the results of the current-state survey, including an analysis of similar existing products and an investigation of user needs for an online programming learning system. Based on these results, the requirements are synthesized and analyzed, and an overview of the main system functionalities is described using use case diagrams. In addition, several representative business processes of the system are presented. Finally, the chapter selects important use cases for detailed specification and identifies non-functional requirements as a basis for system design and implementation.

2.1 Status survey

2.1.1 Survey of Similar Products

Currently, the online education market in Vietnam is clearly differentiated by content operation models. This thesis selects three representative platforms, namely CodeLearn, F8 Education, and Unica, for survey and evaluation.

- **CodeLearn:**

- **Characteristics:** An ecosystem focused on training algorithmic thinking through direct hands-on practice.
- **Strengths:** Integration of a powerful multi-language online compiler (Web IDE) and a professional contest system.
- **Limitations:** Training content is mainly developed by internal experts from FPT Software or contributed by the user community. This leads to a lack of diversity in teaching styles and a shortage of in-depth skill courses provided by independent instructors outside the organization.

- **F8 Education (Fullstack.edu.vn):**

- **Characteristics:** Focuses on building structured learning roadmaps for web developers.
- **Strengths:** High-quality instructional videos, a well-organized learning management mechanism, and an active support community.
- **Limitations:** The system has not yet developed an external instructor network to participate in course creation. All instructional materials and content currently rely entirely on internal resources of the F8 team, which limits scalability in expanding into new technology domains beyond the operating unit's areas of expertise.

- **Unica:**

- **Characteristics:** Operates under a course marketplace model, connecting freelance instructors with learners.
- **Strengths:** A large volume of courses with diverse topics and instructors.
- **Limitations:** Due to its open nature, teaching quality is inconsistent and lacks strict pedagogical control. In particular, the platform completely lacks an online compiler, causing learners to face difficulties when they have to install and configure complex programming environments on their personal computers.

Table 2.1: Comparison of Similar Online Programming Learning Platforms

Criteria	CodeLearn	F8 Education	Unica
Video Lectures	Limited, mainly practice-oriented	Very strong, detailed, high quality	Very strong, diverse instructors
Supplementary Materials	In-depth algorithm-focused reading materials	Materials summarizing knowledge after videos	Diverse learning materials (PDF, Slides)
Web-based Practice	Very strong (multi-language IDE)	Strong (focused on Web Front-end)	Not supported
Progress Management	Strictly score-based / ranking-based	Very strict (course-based)	Loose (self-paced learning)
Learning Roadmap	Based on programming languages	Based on career paths	Unclear
Interaction and Support	Technical forums	Community groups (Discord/FB)	Comments (slow feedback)

2.1.2 User Needs Survey

Based on the analysis of feedback from learners and the requirements of training institutions, this thesis identifies the core needs for the new system as follows:

- **From the Learners' Perspective:**

- **Immediate interaction:** Learners expect an embedded code editor located next to the lecture content, allowing them to practice newly acquired knowledge immediately without the need to install complex development environments.
- **Discipline and learning roadmap:** A clear learning roadmap and re-

minder mechanisms are required to encourage course completion and avoid procrastination.

- **Assessment system:** Quizzes and examinations must closely reflect real-world requirements to enable learners to self-assess their competencies.
- **From the Instructors' and Administrators' (Admin) Perspective:**
 - **Flexible content management:** The ability to easily create and update lectures, coding assignments, manage promotional banners, and administer discount programs.
 - **Monitoring and statistics:** The system should provide visualized data on learners' progress, course revenue, and instructional effectiveness to allow timely adjustments.
 - **Interaction with learners:** Effective tools for answering questions and managing comments in order to enhance training quality.

2.2 Functional Overview

2.2.1 General use case diagram

The overall use case diagram of the system illustrates the interaction relationships between actors and the main functional groups. The system consists of five primary actors: Guest, Learner, Teacher, Teaching Assistant, and Administrator (Admin).

The Guest actor represents users who do not have an account or have not logged into the system. Guests are allowed to perform basic functions to explore the platform:

- **Register / Login:** Create a new account or log into the system in order to access advanced services.
- **Search Courses:** Search for courses based on personal needs and view detailed information about learning roadmaps, content, and learning materials before making an enrollment decision.

The Learner actor is the central user group of the system, directly consuming educational content and performing learning activities:

- **Enroll and Pay for Courses:** Complete payment transactions to gain access to paid learning content.
- **Participate in Online Learning:** Access various types of learning materials, including watching lecture videos, reading instructional documents, completing

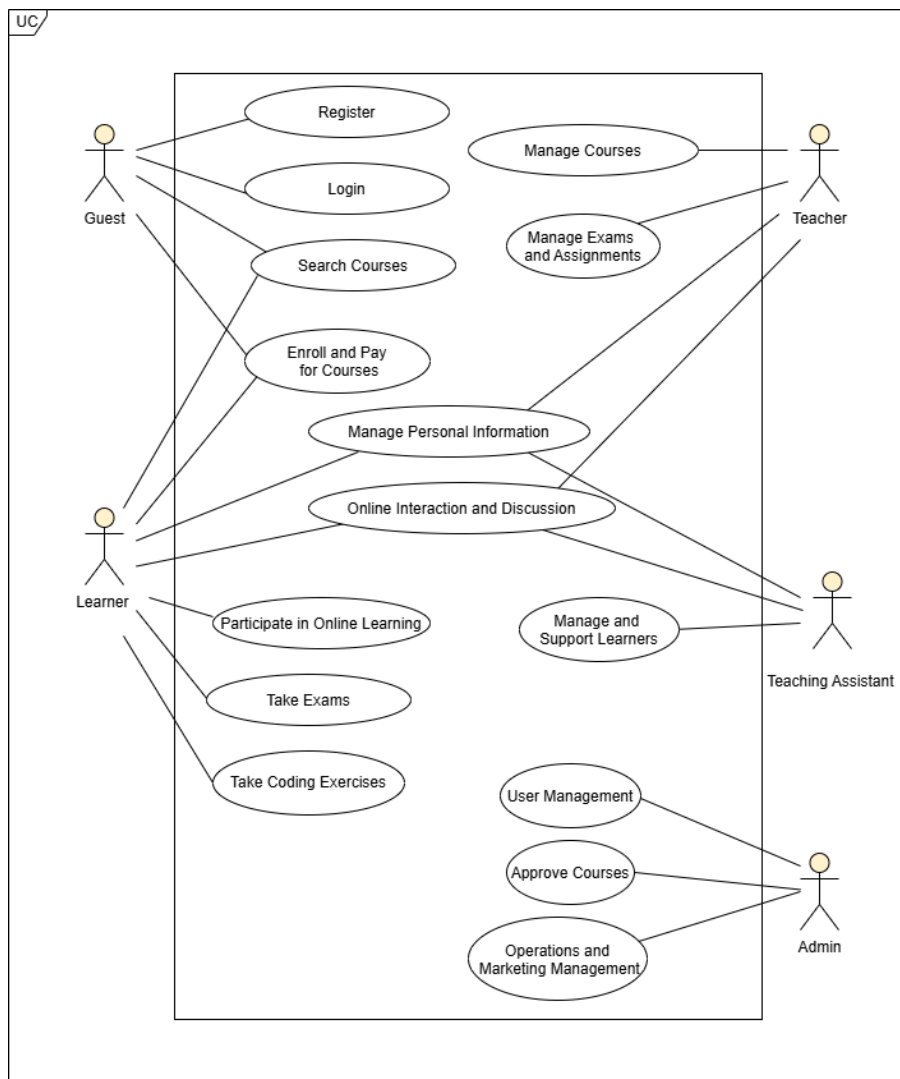


Figure 2.1: General Use Case

quizzes, and performing coding exercises directly in the web browser.

- **Take Exams:** Complete examinations to assess competency levels, evaluate learning outcomes, and perform self-assessment.
- **Take Coding Exercises:** Solve programming problems by writing and refining code to successfully pass all test cases.
- **Online Interaction and Discussion:** Ask questions to Teaching Assistants, participate in discussions through comments, exchange messages with Teachers and Teaching Assistants, and provide feedback on course quality.

The Teacher actor plays a coordinating role and is responsible for providing instructional content to learners throughout the course:

- **Manage Courses:** Create and update course structures, lecture videos, learning materials, and source code content.

- **Manage Exams and Assignments:** Design question banks, programming assignments, and test cases to support automatic evaluation or manual grading.

The Teaching Assistant actor is responsible for supporting learners throughout the training process:

- **Manage and Support Learners:** Both Teachers and Teaching Assistants participate in monitoring learning progress, answering questions, and providing academic feedback to learners.

The Administrator (Admin) actor operates the system at the highest level, ensuring system stability and the legal validity of content:

- **User Management:** Manage the entire user list and assign Teaching Assistants to specific courses to support Teachers.
- **Approve Courses:** Review and approve course content developed by Teachers before it is published on the platform.
- **Operations and Marketing Management:** Manage marketing campaigns through banners and discount programs, and monitor financial and revenue reports of the system.

2.2.2 Detailed use case diagram

a, Detailed Search Courses Diagram

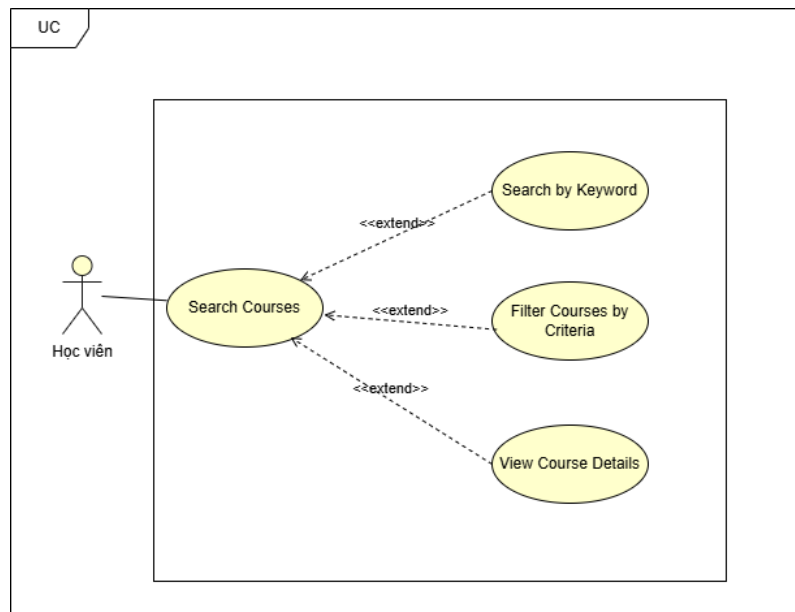
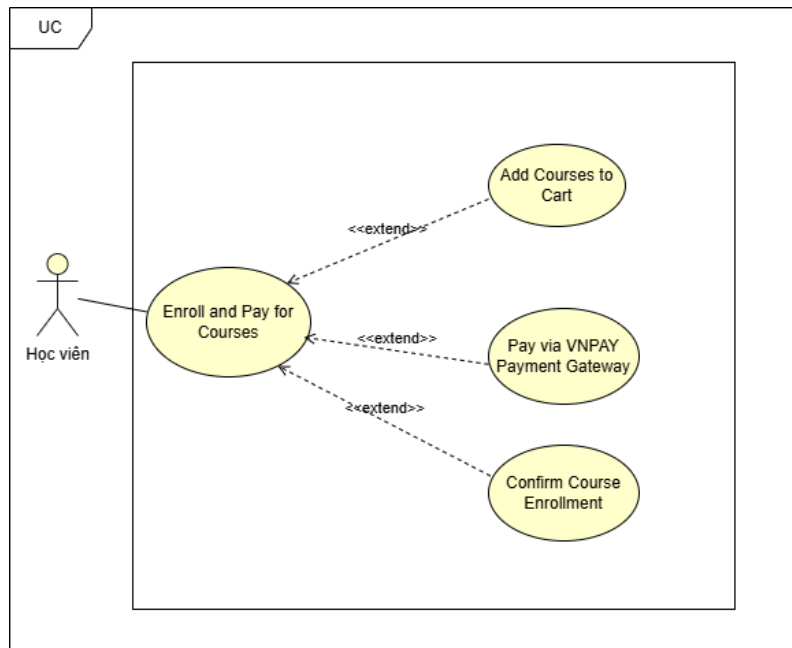
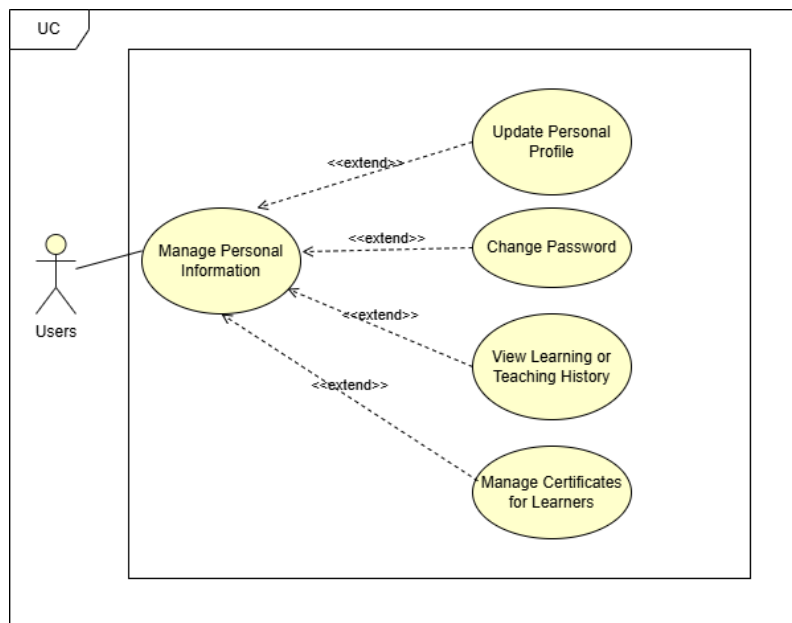
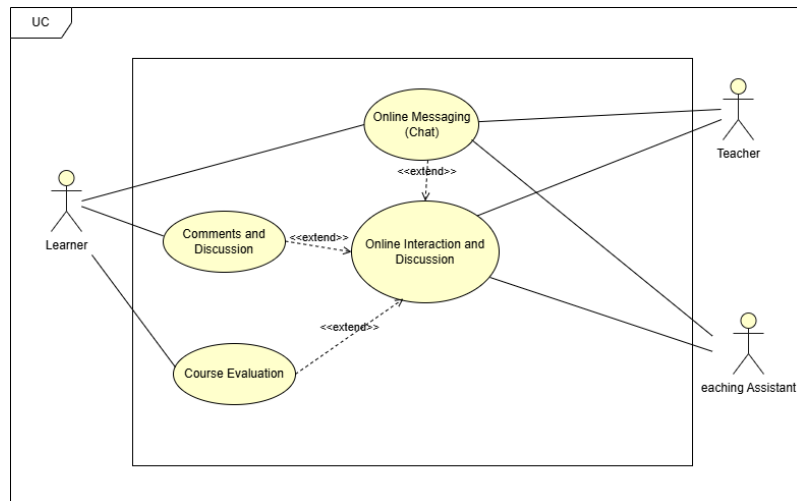
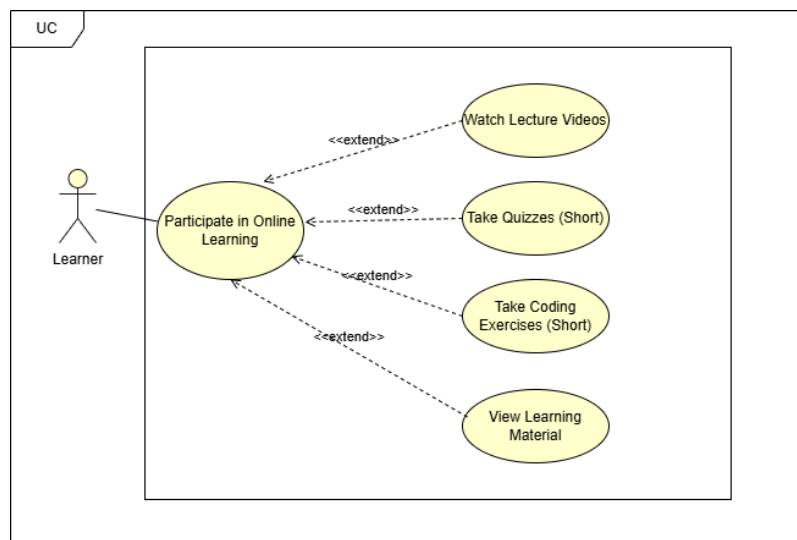
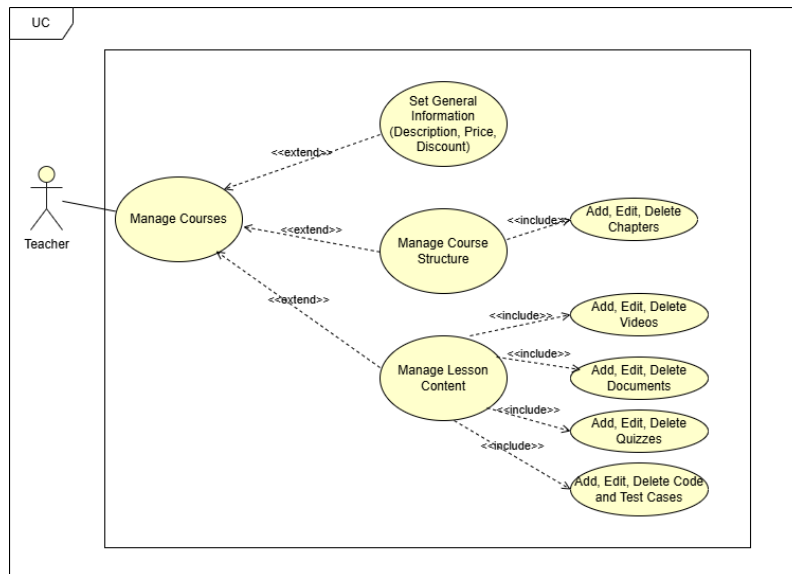
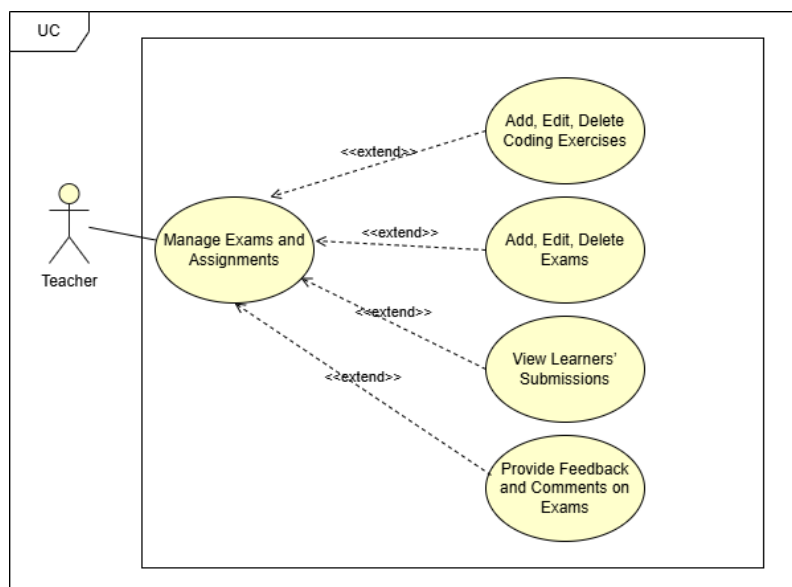


Figure 2.2: Detailed Search Courses Diagram

b, Detailed Enroll and Pay for Courses Diagram**Figure 2.3:** Detailed Enroll and Pay for Courses Diagram**c, Detailed Manage Personal Information Diagram****Figure 2.4:** Detailed Manage Personal Information Diagram

d, Detailed Online Interaction and Discussion Diagram**Figure 2.5:** Detailed Online Interaction and Discussion Diagram**e, Detailed Participate in Online Learning Diagram****Figure 2.6:** Detailed Participate in Online Learning Diagram

f, Detailed Manage Courses Diagram**Figure 2.7:** Detailed Manage Courses Diagram**g, Detailed Manage Exams and Assignments Diagram****Figure 2.8:** Detailed Manage Exams and Assignments Diagram

h, Detailed Manage and Support Learners Diagram

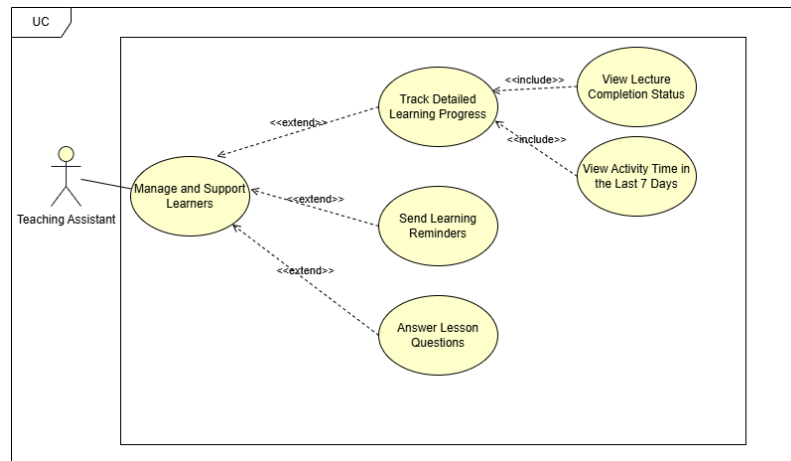


Figure 2.9: Detailed Manage and Support Learners Diagram

i, Detailed User Management Diagram

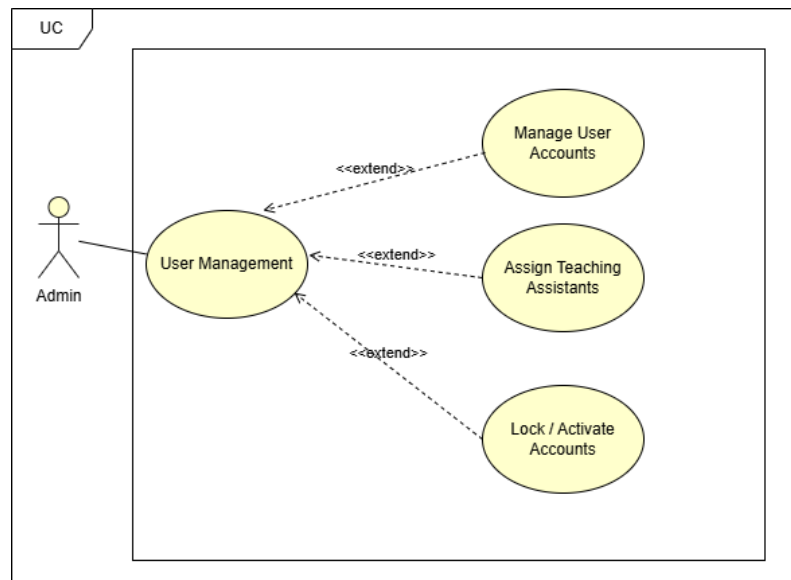
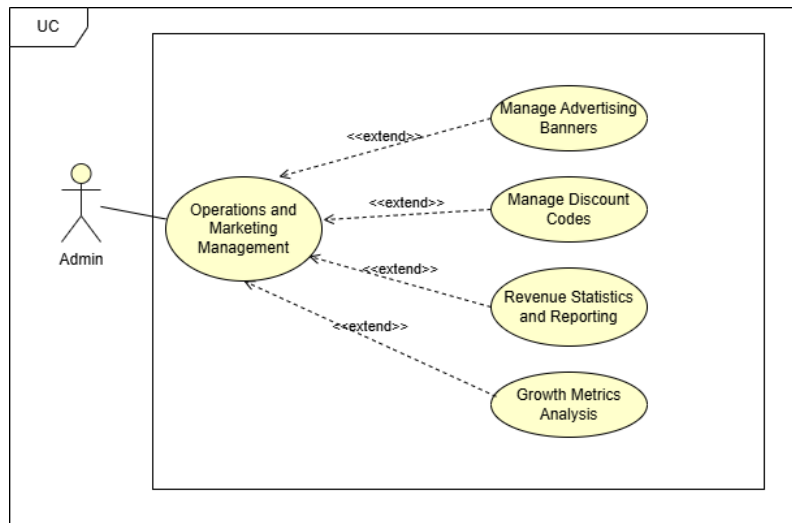


Figure 2.10: Detailed User Management Diagram

j, Detailed Operations and Marketing Management Diagram**Figure 2.11:** Detailed Operations and Marketing Management Diagram**2.2.3 Business process****a, Learning and Programming Practice Process**

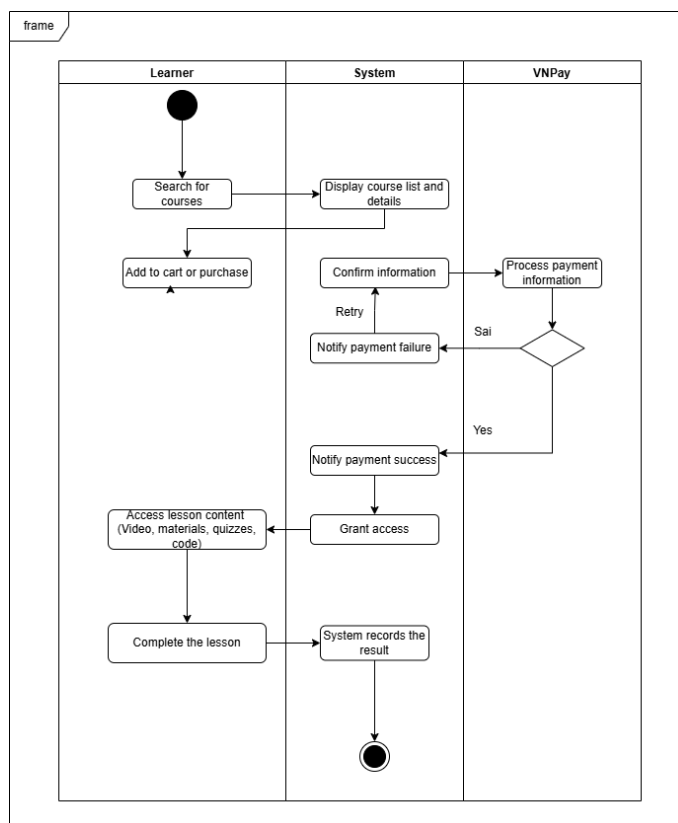


Figure 2.12: Learning and Programming Practice Process

b, Course Content Development and Approval Process

2.3 Functional description

2.3.1 Use Case Specification: Login

Use Case Name	Login
Actors	Learner, Teacher, Teaching Assistant, Administrator (Admin)
Use Case Description	The user uses a registered Email and Password to authenticate and access the system.
Preconditions	The user has successfully registered an account (the account is not locked).

Main Flow	<i>Actor Actions</i> 1. Access the home page and click the “Login” button on the navigation bar. <i>System Responses</i> 2. Display a login form requesting Email and Password. <i>Actor Actions</i> 3. Enter Email and Password, select a role, and click “Login”. <i>System Responses</i> 5. Encrypt the entered password and validate it against the database. 6. If authentication is successful, create a working session (Session/Token). 7. Check the user role (Role). 8. Redirect the user to the corresponding Dashboard (e.g., Learner dashboard, Admin dashboard, etc.).
Alternative Flows	A1. Invalid login information: If, at Step 3, the Email does not exist or the Password is incorrect, the system displays the message “Incorrect email or password” and requests the user to re-enter the information. A2. Account locked: If the account has been locked by the Admin, the system displays the message “Your account has been locked. Please contact support.” A3. Forgot password: The user clicks “Forgot password”, and the system switches to the password recovery process via Email.
Postconditions	The user is successfully logged in, and the system stores the login state so that the user can perform subsequent functions (e.g., purchasing courses, learning, management, etc.).

2.3.2 Use Case Specification: Enroll and Pay for Courses

Use Case Name	Enroll and Pay for Courses
Actor	Learner
Use Case Description	The Learner purchases a course via an online payment gateway in order to obtain access rights to the course learning content.

Preconditions	The Learner has logged into the system and is viewing the detailed information page of a course that the Learner does not yet own.
Main Flow	<i>Actor Actions</i> 1. On the course detail page, click “Buy Now” or “Add to Cart”. <i>System Responses</i> 2. Display the cart interface and payment confirmation page. <i>Actor Actions</i> 3. Review the order information and select the payment method (VNPAY). <i>System Responses</i> 4. Redirect the Learner to the partner payment gateway (VNPAY). <i>Actor Actions</i> 5. Enter card/bank account information on the payment gateway. 6. Authenticate the transaction (OTP / Banking app). <i>System Responses</i> 7. Send the transaction processing request to the bank. 8. Receive the result from the payment gateway, display the message “Payment successful”, and unlock the course.
Alternative Flows	A1. Payment failed: At Step 8, if the payment gateway returns an error (e.g., incorrect information, insufficient balance, etc.), the system displays an error message and requests the Learner to retry the payment or choose another method. A2. Cancel operation: During the payment step, if the Learner clicks “Cancel”, the system returns to the course detail page.
Postconditions	The course is added to the “My Courses” list, and the order status is updated to “Paid”.

2.3.3 Use Case Specification: Take Coding Exercises

Use Case Name	Take Coding Exercises
Actor	Learner
Use Case Description	The Learner practices writing source code directly in the web browser to solve programming problems and receives results based on passing automated test cases.

Preconditions	The Learner has accessed a practice lesson (“Practice Exercise” lesson type).
Main Flow	<i>Actor Actions</i> 1. Read the problem statement and the input/output requirements. <i>System Responses</i> 2. Display the web-based code editor (Web IDE) next to the problem statement. <i>Actor Actions</i> 3. Write source code in the web editor. 4. Click the “Run” button. <i>System Responses</i> 5. Receive the source code and perform compilation. 6. Execute the program against hidden test cases (test scenarios). 7. Compare the actual output with the expected output. 8. Display the message “Pass” if all test cases are correct.
Alternative Flows	A1. Compile Error: The system displays detailed syntax error lines so that the Learner can fix the code. A2. Wrong Answer: The system displays the message “Wrong”, and provides information about the failed test case so that the Learner can debug the algorithm.
Postconditions	The practice exercise status is updated to “Completed”, and the score is recorded in the learning progress tracking.

2.3.4 Use Case Specification: Manage Courses

Use Case Name	Manage Courses
Actor	Teacher
Use Case Description	The Teacher creates a new course, composes course content and syllabus structure, and uploads learning materials to the course.
Preconditions	The Teacher has logged into the admin dashboard (teacher area).

Main Flow	<i>Actor Actions</i> 1. Select the function “Create a New Course”. <i>System Responses</i> 2. Display a form for entering general information (Name, Price, Description). <i>Actor Actions</i> 3. Enter general information (name, description, price, cover image). 4. Click “Save”. 5. Send a request to Admin for approval. 6. Create the course module structure (Module). For each module, add lessons (Video, Documents, Coding Exercises). <i>System Responses</i> 7. Save the information and display the interface for adding lessons. <i>Actor Actions</i> 8. Add a YouTube URL, upload documents, create coding exercises, or add quizzes into the lecture content. 9. Click “Save”. <i>System Responses</i> 10. Save the data to the database.
Alternative Flows	A1. Invalid data: If the Teacher does not fill in required fields, the system displays an error message and requests the Teacher to complete the information before allowing submission for approval.
Postconditions	The course is stored in the system.

2.3.5 Use Case Specification: Approve Courses

Use Case Name	Approve Courses
Actor	Administrator (Admin)
Use Case Description	The Admin reviews the course content submitted by the Teacher and decides whether to publish the course or reject it.
Preconditions	There is at least one course with the status “Pending Approval” in the list.

Main Flow	<i>Actor Actions</i> 1. View the list of courses pending approval. <i>System Responses</i> 2. Display the list of courses that require processing. <i>Actor Actions</i> 3. Select a course to review detailed content (videos, assignments). <i>System Responses</i> 4. Display the full course content for Admin preview. <i>Actor Actions</i> 5. Click the “Approve” button. <i>System Responses</i> 6. Update the course status to “Published”. 7. Display the course on the homepage and send a notification to the Teacher.
Alternative Flows	A1. Reject approval: The Admin clicks the “Reject” button and enters a reason or required revisions. The system changes the status to “Revision Requested” and sends a notification containing the reason to the Teacher.
Postconditions	The course status is updated according to the Admin’s decision.

2.3.6 Use Case Specification: Online Interaction and Discussion

Use Case Name	Online Interaction and Discussion
Actors	Learner, Teaching Assistant
Use Case Description	The Learner submits questions related to a lesson, and the Teaching Assistant provides explanations and answers.
Preconditions	The Learner is studying a paid lesson and encounters an issue requiring support.

Main Flow	<i>Actor Actions (Learner)</i> 1. On the lesson interface, click the “Ask Teaching Assistant” button. <i>System Responses</i> 2. Store the question, display it in the discussion list, and notify the Teaching Assistant. <i>Actor Actions (Learner)</i> 3. Enter the question content (optionally attach an image or a code snippet). 4. Click “Send Question”. <i>System Responses</i> 5. Display the question in the discussion list. 6. Send a notification to the assigned Teaching Assistant. <i>Actor Actions (Teaching Assistant)</i> 7. Receive the notification and view the question content. 8. Enter the response or guidance. 9. Click “Send Response”.
Alternative Flows	A1. Delete comment: If the content violates community standards, Admin or the Teaching Assistant deletes the comment.
Postconditions	The question and discussion content is stored so that other learners can participate in the discussion.

2.3.7 Use Case Specification: User Management

Use Case Name	User Management
Actor	Administrator (Admin)
Use Case Description	The Admin manages the user list, performs role assignment and assigns Teaching Assistants to courses.
Preconditions	The Admin has logged into the back-office (admin dashboard).

Main Flow	<i>Actor Actions</i> 1. Select the function “Assign Teaching Assistants”. <i>System Responses</i> 2. Display the Teaching Assistant assignment interface. <i>Actor Actions</i> 3. Select a Teaching Assistant and select the course to be supported. 4. Confirm the assignment. <i>System Responses</i> 5. Update the Teaching Assistant’s access permissions for the selected course. 6. Display the message “Update successful”.
Alternative Flows	A1. Lock account: If the Admin detects a policy violation, the Admin selects the function “Lock account”. The system disables the user’s login access for that account.
Postconditions	User permissions and user status are updated in the database.

2.4 Non-functional requirement

The system must satisfy the following quality standards to ensure stable operation, security, and the best possible user experience.

2.4.1 Performance

- **Response time:** Basic operations such as page loading and course search must respond within 3 seconds.
- **Code execution speed:** The automatic grading module must return results (compilation and test case execution) within 5–10 seconds to maintain learners’ thinking flow.
- **Video display optimization:** Since the system uses embedded (YouTube) videos, the interface must apply asynchronous loading techniques (lazy loading) for video frames (iframes), ensuring smooth video rendering without slowing down the overall page loading speed.

2.4.2 Reliability

The system must ensure stable operation and data preservation:

- **Availability:** Operate continuously 24/7 with an uptime of at least 99.9%.
- **Data safety:** Establish an automatic daily backup mechanism and a rapid re-

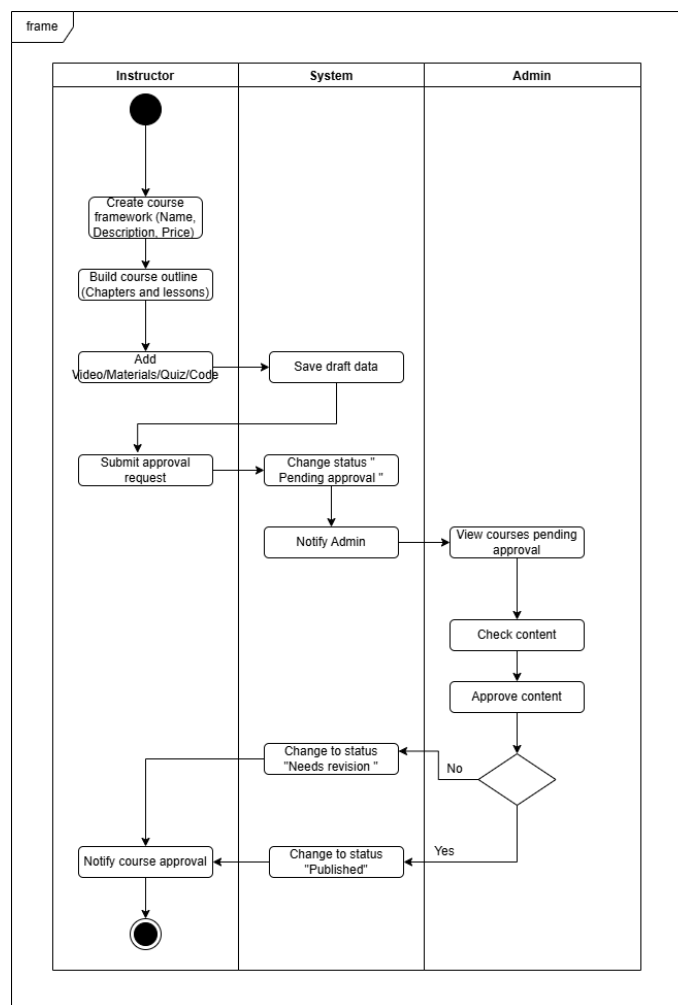


Figure 2.13: Course Content Development and Approval Process

covery process to minimize risks in case of incidents.

- **Response performance:** Optimize application performance to ensure user interactions are always smooth, without annoying delays.

2.4.3 Usability

- **Intuitive interface:** Provide a user-friendly design focusing on the coding exercise experience (code editor) and lecture viewing.
- **Programming support:** The web-based code editor must support syntax highlighting and clear line-level error reporting.
- **Compatibility:** The system must display well on web desktop platforms and common browsers (Chrome, Edge, Firefox).

2.4.4 Security

- **Information security:** User passwords are hashed before being stored.
- **Payment safety:** Integrate the VNPay gateway according to security stan-

dards, and do not store users' bank card information directly in the system.

- **Authorization:** Strictly control access permissions among Learners, Teachers, and Admin.

2.4.5 Maintainability

The system is designed for convenient long-term management and development:

- **Source code quality:** The source code must be well organized and strictly follow programming standards to facilitate inspection and debugging.
- **Documentation:** Provide complete and detailed technical documentation as a reference foundation for future maintenance and system upgrades.

2.4.6 Scalability

The system architecture is ready to support future growth:

- **Feature scalability:** A flexible design allows easy integration of new functional modules without affecting the existing system.
- **System scalability:** The system can adapt to increasing numbers of users through infrastructure upgrades and software optimization.

CHAPTER 3. METHODOLOGY

3.1 Technologies and Platforms

In this chapter, I present the technologies and how they are applied to the features of the system. These technologies are selected based on their suitability for the problem requirements, in order to ensure that the system operates smoothly, accurately, and can be upgraded easily.

3.1.1 Java Programming Language

Java is a high-level, object-oriented programming language developed by Sun Microsystems (currently owned by Oracle) and first released in 1995. With the well-known philosophy “Write Once, Run Anywhere” (WORA), Java has become one of the most popular languages in the world, especially for building enterprise systems, large-scale web applications, and mobile applications.

a, Syntax and features

Java stands out for its strictness, security, and high performance. The following are key features that make Java a top choice for large software systems:

- **Platform independence:** Thanks to the Java Virtual Machine (JVM), compiled Java code (bytecode) can run on any operating system (Windows, macOS, Linux) without recompilation.
- **Fully object-oriented:** Everything in Java revolves around objects, which helps manage, reuse, and extend code effectively, and fits complex software design models.
- **Multithreading:** Java provides built-in multithreading support, enabling multiple tasks to be executed concurrently. This helps optimize performance for applications that handle heavy workloads such as code grading systems or payment processing.
- **Automatic memory management:** Java uses Garbage Collection to automatically release unused memory, reducing memory leaks and improving program stability.
- **Rich standard library:** Java provides a large API set (Java SE API) supporting database connectivity (JDBC), networking, and security.

b, Applications of Java

Java is widely applied in many important technology domains, including:

- **Web backend development:** Java is a leading language for backend development of large systems with powerful frameworks such as Spring Boot and Jakarta EE.
- **Enterprise applications:** Banking systems, e-commerce platforms, and enterprise resource planning (ERP) systems are often built with Java due to its high security and stability.
- **Big Data processing:** Popular Big Data technologies such as Apache Hadoop and Apache Spark are written in or strongly support Java.
- **Android applications:** Java used to be the official language for Android development before Kotlin emerged (and it is still fully compatible).

c, Community and documentation

Java has one of the largest developer communities in the world after more than two decades of development. Official documentation from Oracle is comprehensive and detailed. In addition, forums such as StackOverflow and open-source repositories on GitHub provide strong support for virtually any technical issue.

With the requirement to build a programming management and teaching system that needs high stability, strong concurrency handling, and a strict structure for maintainability, Java is an optimal choice. In particular, the combination of Java and the Spring Boot Framework shortens development time while ensuring system performance and security.

3.1.2 Spring Boot Framework

Spring Boot **springboot** is a powerful open-source framework developed by the Pivotal Team (currently under VMware) based on the Spring Framework. Spring Boot provides a solution to quickly build stand-alone and production-grade applications. With the philosophy “Convention over Configuration”, Spring Boot minimizes complex setup, allowing developers to focus on implementing business logic effectively.

a, Architecture

Spring Boot architecture is commonly built following a layered architecture. In this model, system components are clearly separated by responsibilities as follows:

- **Controller class (Presentation layer):** Acts as the entry point for client requests. These classes define API endpoints to receive data and forward requests to the business layer.
- **Service class (Business layer):** Contains the core business logic of the sys-

tem. This layer receives requests from controllers, performs computations or business workflows, and interacts with the data layer.

- **Repository interface (Data access layer):** Responsible for communicating with the database. This layer abstracts SQL operations and provides methods for data manipulation (CRUD).
- **Model (Entities):** Objects representing the data structure in the system, used for transforming and storing information between layers and the database.

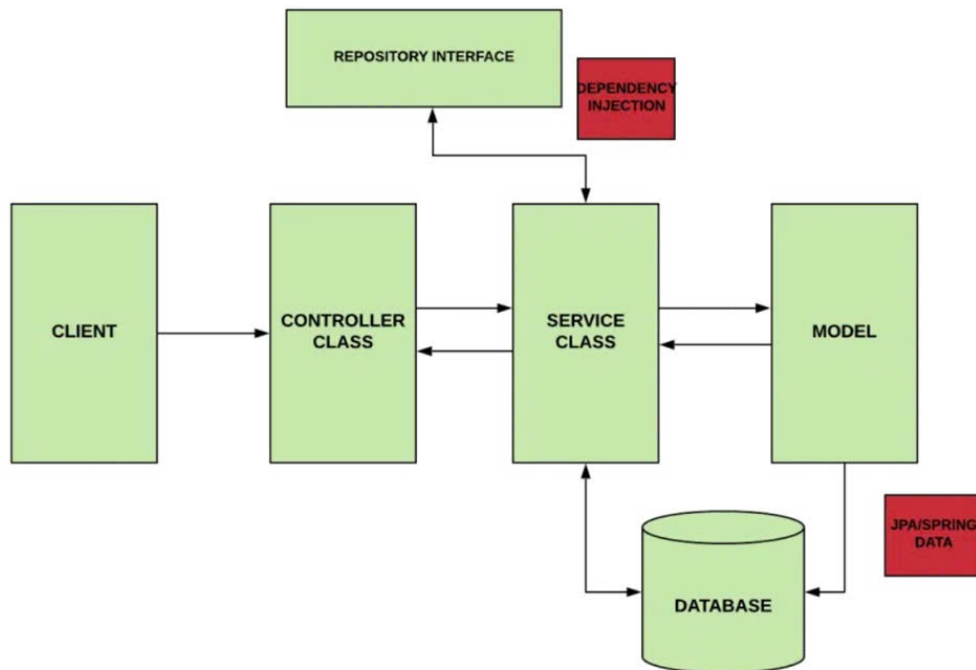


Figure 3.1: Spring Boot Layered Architecture

b, Key components

- **Dependency Injection (DI):** As illustrated in Figure 3.2 (red box), Spring Boot uses DI to manage dependencies between classes (Controller calls Service, Service calls Repository) without manual object instantiation, enabling loose coupling and better testability.
- **JPA / Spring Data:** This component (top-right red box) provides Object-Relational Mapping (ORM), automatically mapping models to database tables and vice versa.
- **Database:** The physical data storage of the system (MySQL is used in this project), interacting directly with the Repository layer.

c, Features

Spring Boot provides additional features to support modern web application development, including:

- **Auto-configuration:** Spring Boot automatically configures the application based on dependencies added by the developer. For example, when a database library is included, Spring Boot automatically sets up required beans without complex XML configuration.
- **Embedded server:** Spring Boot embeds web servers such as Tomcat, Jetty, or Undertow inside the application, allowing it to run as a standalone Java process without installing a separate server.
- **Spring Security:** Provides strong authentication and authorization mechanisms, protecting APIs and ensuring only authorized users can access sensitive resources.
- **Spring Data JPA:** Simplifies database operations by generating queries from method names, reducing boilerplate code.
- **Actuator:** Provides endpoints for monitoring and managing the running application (health checks, metrics), enabling developers to track system status easily.

These features make Spring Boot a solid platform that meets the requirements of performance, security, and scalability for a programming management and teaching system.

3.1.3 ReactJS

ReactJS **react** (often called React) is an open-source JavaScript library for building user interfaces, developed by Facebook (now Meta). First released in 2013, ReactJS quickly became one of the most popular tools for building Single Page Applications (SPA). React enables developers to create reusable UI components, making code development and maintenance more flexible and efficient.

a, Architecture

ReactJS is based on a component-based architecture, where the user interface is divided into independent parts called components. Each component can manage its own state and can be combined to build complex interfaces.

- **Component layer:** Where developers write code using JSX (JavaScript XML). Components define the structure and UI of the application. There are two main types: Function Components (simple, stateless or using Hooks) and Class

Components (stateful with lifecycle methods).

- **Virtual DOM:** The core of React architecture. It is a lightweight copy of the real DOM stored in memory. When the state changes, the Virtual DOM updates without immediately affecting the UI directly.
- **Reconciliation and renderer:** React uses a diffing algorithm to compare the new Virtual DOM with the previous version. After identifying changed elements, React updates only those elements in the Real DOM via ReactDOM, optimizing UI re-rendering performance.

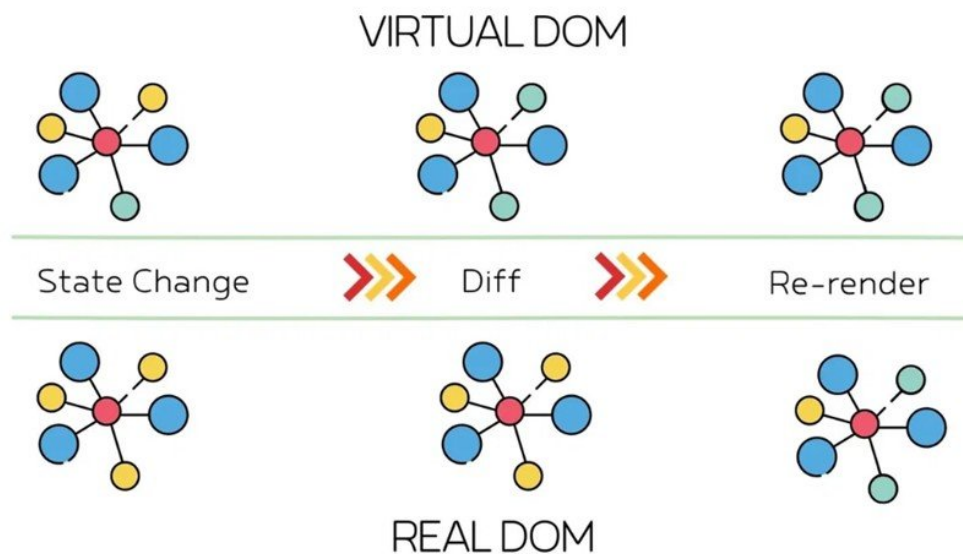


Figure 3.2: ReactJS Virtual DOM Architecture

b, Features

ReactJS provides many advanced features that optimize developer experience and application performance:

- **Virtual DOM:** React updates UI efficiently by updating only necessary elements instead of reloading the entire page, improving smoothness and saving browser resources.
- **Component-based:** Encourages decomposing UI into reusable independent components, making code cleaner, easier to manage, and easier to maintain or upgrade.
- **JSX:** Allows writing HTML structure inside JavaScript, improving readability and stronger integration between UI structure and logic.
- **One-way data binding:** Data flows in a single direction from parent to child via props, making data flow predictable and easier for debugging and state control.

- **Rich ecosystem:** React has a large community and diverse libraries (e.g., Redux for state management, React Router for navigation), supporting most needs in modern web development.

3.1.4 MySQL Database

MySQL **mysql** is the most popular open-source relational database management system (RDBMS) today, owned by Oracle Corporation. MySQL is known for fast processing speed, high reliability, and its client-server model. It is an optimal data storage solution for modern web applications, ranging from personal projects to large-scale enterprise systems.

a, Architecture and features

MySQL operates on a client-server model with a strict layered processing architecture. Its architecture consists of three main layers: the Connection Layer responsible for authentication and security; the SQL Layer handling parsing, query optimization, and cache management; and the Storage Engine Layer, which supports flexible choices of engines such as InnoDB or MyISAM.

Key features of MySQL include strict compliance with ACID to ensure integrity for critical transactions, high performance through efficient indexing, multi-layer privilege and security mechanisms, and scalability through replication and clustering.

3.1.5 Other Technologies

In addition to the core technologies, the system integrates third-party services to optimize user experience and provide a safe and effective programming practice environment.

a, YouTube IFrame Player API

Introduction. The YouTube IFrame Player API is an application programming interface provided by Google that allows developers to embed the YouTube video player directly into a website through the `<iframe>` tag. Instead of storing large video files on a private server, this technology enables the system to display and control videos from YouTube using only a URL, saving storage resources and bandwidth.

Key features. The YouTube IFrame Player API provides strong interaction capabilities, including:

- **Embed and play videos:** Display high-quality videos on the website via YouTube video ID or URL, with automatic resolution adjustment based on the user's network speed.

- **UI customization:** Provide parameters to customize the player, such as showing/hiding controls, disabling related video suggestions, and customizing the progress bar color.
- **JavaScript control:** Allow web applications to interact with the player programmatically, supporting actions such as autoplay, pause, seeking, and volume adjustment.
- **Event handling:** Provide event listening to track learner status, for example detecting when a video ends to automatically mark the lesson as completed.

b, Judge0 API

Introduction. Judge0 is a powerful and secure open-source online code execution system. It provides a backend RESTful API to compile and run user code remotely. In this project, Judge0 acts as a “judge”, receiving source code from the website, executing it in an isolated environment, and returning test results.

Key features. Judge0 provides an automatic grading platform with the following notable features:

- **Multi-language support:** Supports compilation and execution of more than 50 popular programming languages (including Java, Python, C++, C#), enabling system flexibility for course expansion.
- **Sandboxing execution environment:** Ensures system safety by running user code in containers with restricted permissions, preventing destructive actions or unauthorized access to server resources.
- **Standard input/output management:** Supports loading standard input (`stdin`) and collecting standard output (`stdout`) or errors (`stderr`) to compare with sample test cases.
- **Performance measurement:** Provides detailed execution time (CPU time) and memory usage information, supporting evaluation of learners’ algorithm efficiency.

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

4.1 Architecture design

4.1.1 Software architecture selection

The architecture selected to develop the system is the Client–Server model combined with a RESTful API architecture. This is a standard model in modern web application development, which completely separates the user interface (Frontend) from business logic processing (Backend), ensuring scalability and ease of maintenance.

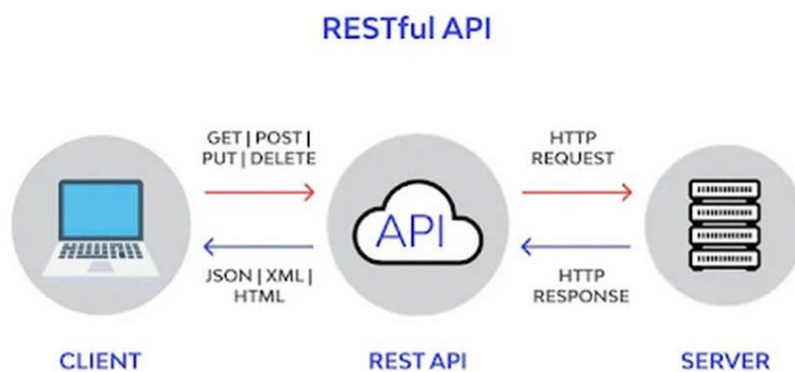


Figure 4.1: Client–Server Architecture Model (RESTful API)

Introduction to the Client–Server architecture (RESTful API). In this model, the system is divided into two independent functional blocks. The client sends HTTP requests (GET, POST, PUT, DELETE) to the server. The server receives the requests, processes business logic, accesses the database, and returns results in the standard JSON format. This allows the client to focus only on presentation and user experience, while the server focuses on security and data processing.

Applying the architecture to the project. The above model is applied to build the “Online Programming Management and Teaching System”. The main components of the system architecture include:

Client:

- **Web Application (ReactJS):** This is the only interaction interface for all user roles (Learner, Teacher, Admin).
- **For Learners:** Provide interfaces for watching lecture videos (YouTube em-

bedded), completing quizzes, and especially an intuitive code editor (Code Editor) for practice.

- **For Teachers/Admin:** Provide dashboards to manage courses, track learner progress, and view revenue statistics reports.

Server:

- **Backend (Spring Boot):** Acts as the core of the system. Spring Boot provides RESTful APIs to serve the client, handling complex business logic such as user authentication (Spring Security), order management, and orchestrating the grading process.
- **Database (MySQL):** Responsible for persistently storing all system data, including user information, lesson content, submitted source code, and transaction history.
- **Grading Service (Judge0 API):** An independent integrated service to compile and execute learners' source code in a secure sandbox environment, then return the output to the backend for grading processing.
- **Video Service (YouTube API):** The system does not store videos directly; instead, it uses YouTube infrastructure to stream lecture videos, optimizing bandwidth usage and viewing experience.
- **Payment (VNPay):** Integrates a payment gateway to process course purchase transactions safely and automatically.

4.1.2 Overall design

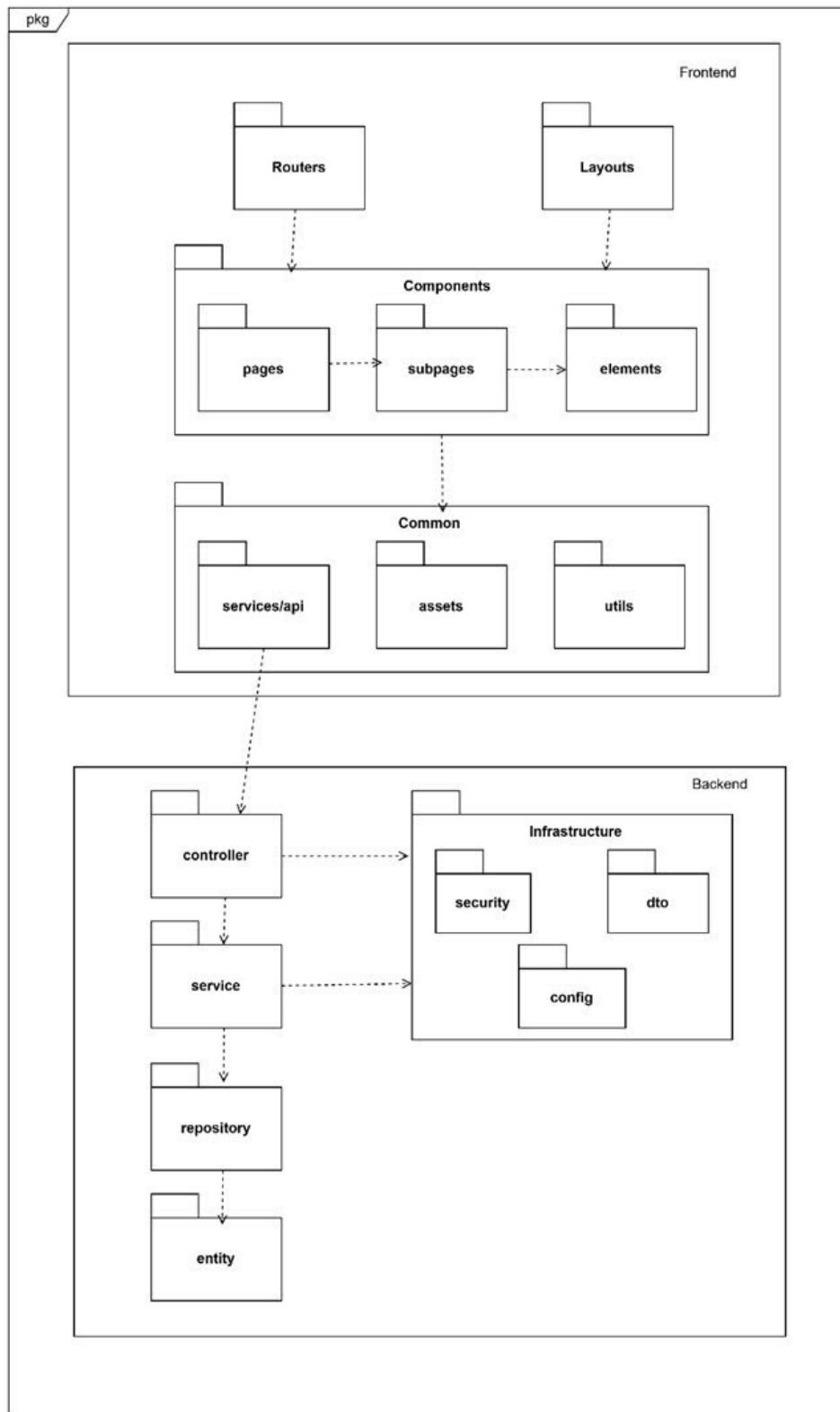


Figure 4.2: System Overview Diagram

Detailed Explanation of System Packages **Frontend (User Interface)**. The Frontend consists of a set of components that build an intuitive interactive interface for

users. The Frontend structure includes the following main packages:

- **App**: Acts as the core organizer, managing the application layout and navigating between application screens.
- **Components**: Contains small UI components (such as buttons, forms, and tables) or reusable display logic blocks that are composed to form a complete page.
- **FrontendService**: An intermediate service package, including:
 - **HTTP**: Provides the connection protocol, sends requests to the Backend API, and handles response data.
 - **Helper**: Defines helper functions and shared constants used throughout the application.
- **Utils**: Provides common utility functions (string processing, date/time formatting) used to build logic for screens.
- **Environments**: Stores system environment configuration information such as environment variables, secret keys, and the connection address to the Backend server.
- **Assets**: A repository of static resources such as images and icons used in the interface.
- **Styles**: A collection of styling files (CSS/SCSS) that standardize the interface, define colors, typography, and visual effects for the application.

Backend (Business Logic Processing). The Backend consists of components responsible for processing core logic and managing data:

- **Views (Controller)**: Receives requests from the Frontend and coordinates the returned data.
- **URLs**: A routing system that manages endpoints to receive requests from the Client.
- **Config**: Contains important connection configurations to the MySQL database (on AWS RDS) and integrates third-party services such as Google Cloud.
- **Models**: Directly maps to database tables, supporting query and storage operations (ORM).
- **Utils**: Contains shared utility functions for internal server-side data processing.
- **Constant**: Defines system constants, error codes, or standardized Backend

messages.

- **Templates (DTO):** Defines standardized data structures (Data Transfer Objects) for API responses to ensure consistency when communicating with the Frontend.

4.1.3 Detailed package design

Sinh viên thiết kế và lần lượt vẽ biểu đồ thiết kế cho từng package, hoặc một nhóm các package liên quan để giải quyết một vấn đề gì đó. Khi vẽ thiết kế gói, sinh viên chỉ cần đưa tên lớp, không cần chỉ ra các thành viên phương thức và thuộc tính. SV tham khảo ví dụ minh họa trong Hình 4.3.

Sinh viên cần vẽ rõ ràng quan hệ giữa các lớp trong biểu đồ. Các quan hệ bao gồm: phụ thuộc (dependency), kết hợp (association), kết tập (aggregation), hợp thành (composition), kế thừa (inheritance), và thực thi (implementation). Các quan hệ này đều đã được minh họa trong 4.3.

Sau khi vẽ hình minh họa, sinh viên cần giải thích ngắn gọn về thiết kế của mình.

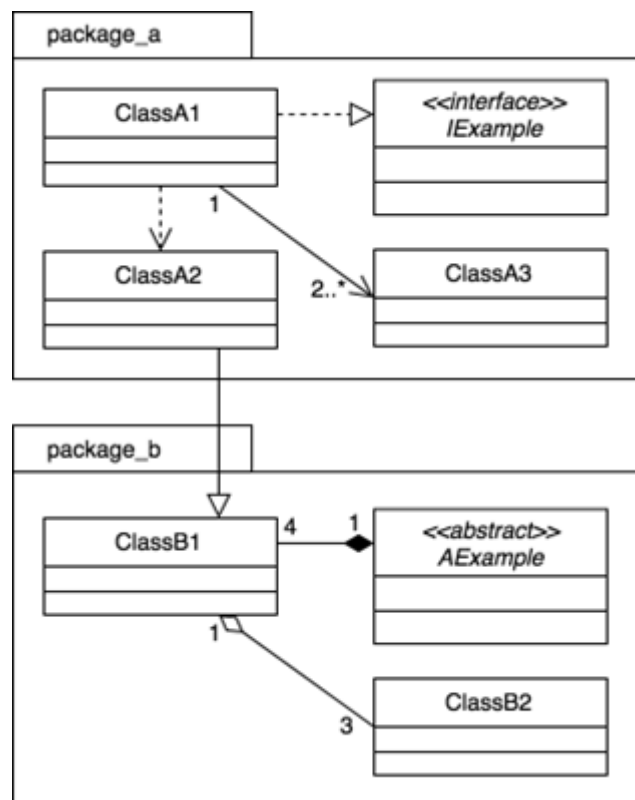


Figure 4.3: Ví dụ thiết kế gói

4.2 Detailed design

4.2.1 User interface design

The user interface of the online programming management and teaching system is built based on the Mobile-first principle, ensuring consistency and an optimal user experience across multiple platforms. The system uses ReactJS in combination with Tailwind CSS and the Ant Design library to implement the following design standards.

Screen Information Specification The system supports responsive display across standard device ranges, as shown in Table 4.1.

Table 4.1: Screen Information Specification

No.	Device	Size	Supported Resolutions
1	Computer (Desktop/Laptop)	11 – 21+ inch	HD (1366x768), Full HD (1920x1080), 2K, 4K
2	Tablet	7 – 12 inch	HD (1280x720), Full HD (1920x1080)
3	Smartphone	5.8 – 8 inch	HD (1280x720), Full HD (1920x1080)

UI Component Standardization Interactive UI components are standardized in terms of size, color, and operating states.

Buttons are designed with rounded corners (`rounded-lg`) and a minimum size of 44x44px to meet accessibility standards. Table 4.2 describes the button design standardization.

Table 4.2: Button Design Standardization

Attribute	Detailed Description
Size	Standard padding: 12px–16px (primary button), 8px–12px (secondary button).
Colors	- Primary button: Blue–Purple gradient (<code>#2563eb</code> to <code>#7c3aed</code>) or Blue (<code>#2563eb</code>). - Secondary button: Gray (<code>#6c757d</code>) or white border. - Warning button: Red (<code>#ef4444</code>).
Effects	Hover: darker color, shadow (<code>shadow-md</code>), slight movement (<code>-translate-y-0.5</code>).

Controls Input elements follow standardized sizes for easier interaction. Table 4.3 describes the control design standardization.

Table 4.3: Control Design Standardization

Control Type	Attribute	Description
Input / Textbox	Size	Minimum 150x40 px. Padding: 12px.
	State	- Normal: light gray border (#d1d5db). - Focus: blue border and ring (#2563eb).
Dropdown / Select	Style	Integrated with Ant Design, rounded corners (rounded-lg), and consistent height with Input.
Checkbox / Radio	Size	20x20 px. Blue color when selected (Active).

Feedback Messages The system uses the Ant Design message component to display status notifications at the top-center of the screen. Table 4.4 specifies feedback message rules.

Table 4.4: Feedback Message Rules

Message Type	Representative Color	Hex Code
Success	Green	#28a745 (Background: #f0fdf4)
Error	Red	#dc3545 (Background: #fef2f2)
Warning	Yellow	#ffc107 (Background: #fffbeb)
Information	Blue	#17a2b8 (Background: #eff6ff)

Color System and Typography The interface uses the Poppins font as the primary typeface, combined with a high-contrast color palette to optimize readability. Table 4.5 presents the UI color palette.

Table 4.5: UI Color Palette

Role	Color	Hex Code	Usage
Primary	Blue (Indigo)	#2563eb	Primary buttons, links, important icons.
Secondary	Purple	#7c3aed	Supporting elements, secondary buttons.
Accent	Red	#ef4444	Delete actions, critical warnings.
Text Main	Black / Dark Gray	#1f2937	Headings, lesson content.
Background	White / Light Gray	#ffffff / #f9fafb	Page background and content blocks.

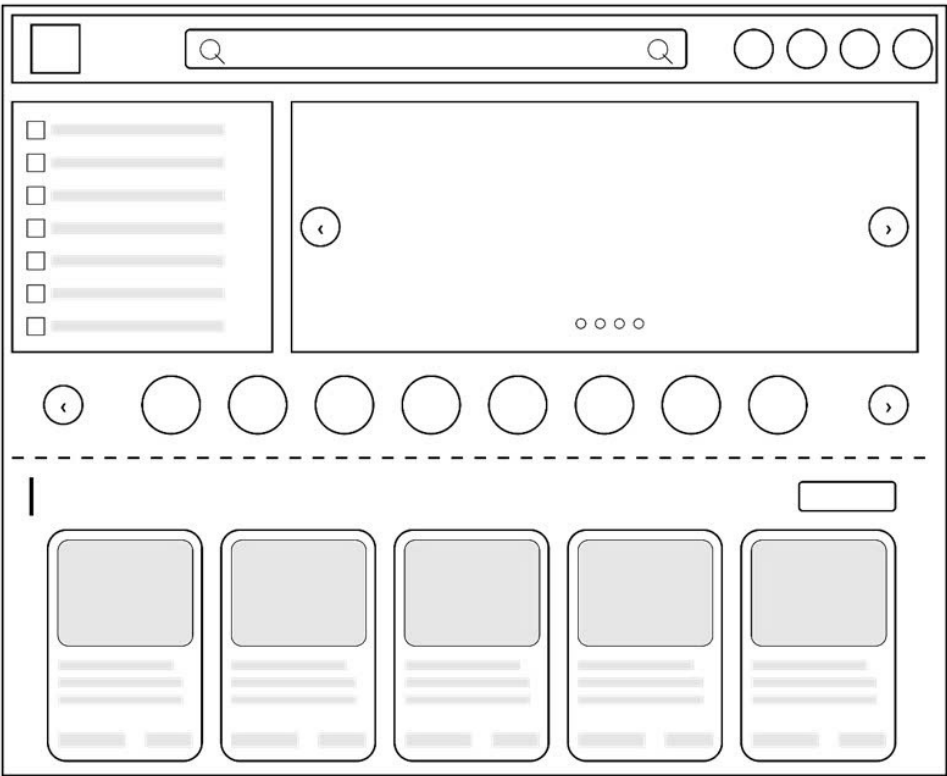


Figure 4.4: Homepage UI Wireframe

The home page wireframe, as illustrated in Figure 4.x, presents the overall layout of the system’s main screen. The interface includes a top bar with search and user functions, a navigation area for accessing main categories, and a central section displaying featured courses in the form of course cards.

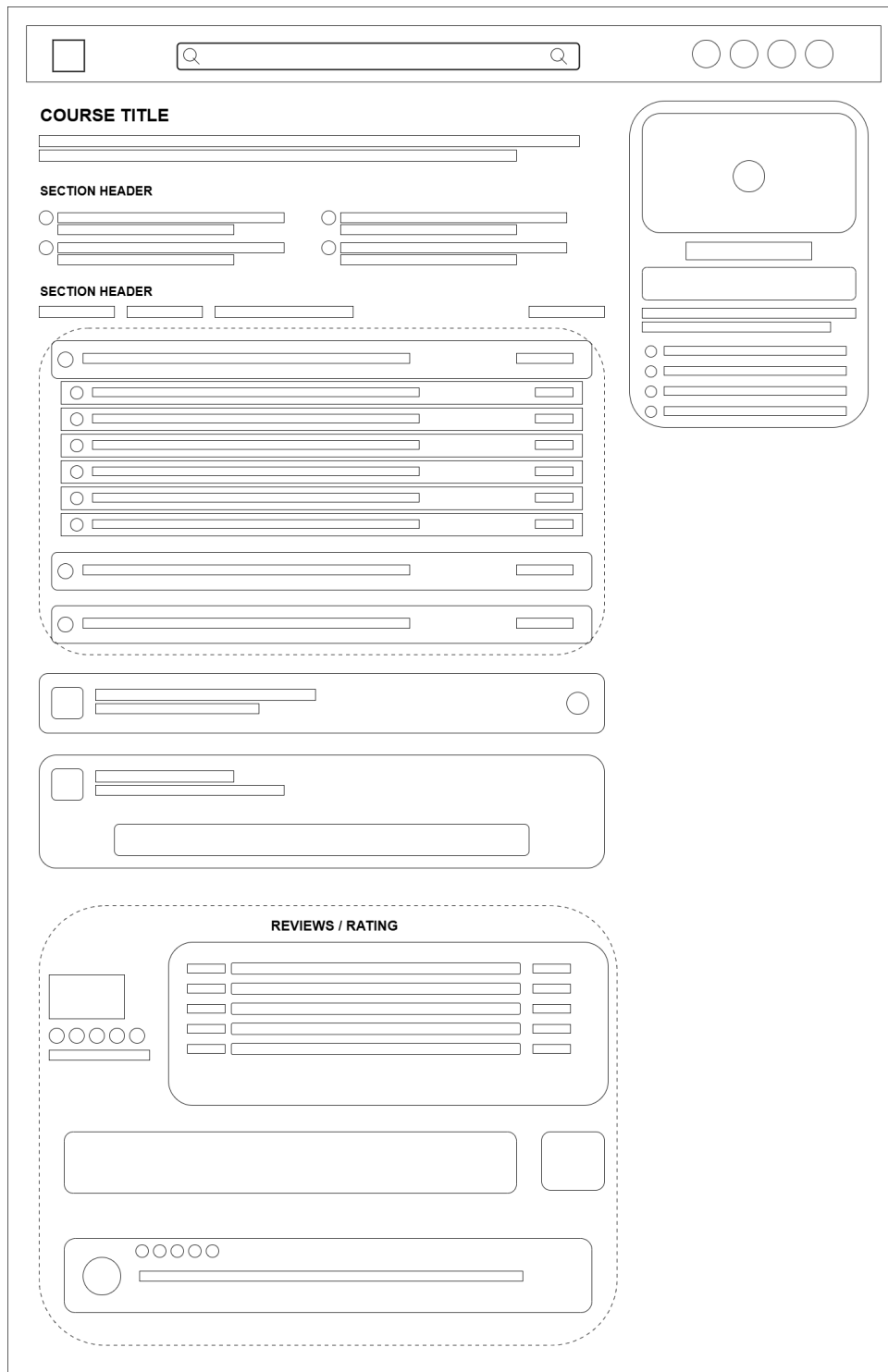


Figure 4.5: Course Detail Page UI Wireframe

The course detail wireframe, as illustrated in Figure 4.x, shows the structure of the course detail screen. The interface displays general course information, a navigation section for lessons, exams, and coding exercises, and a content area for presenting the selected materials. In addition, rating and comment sections are included to support user interaction and feedback.

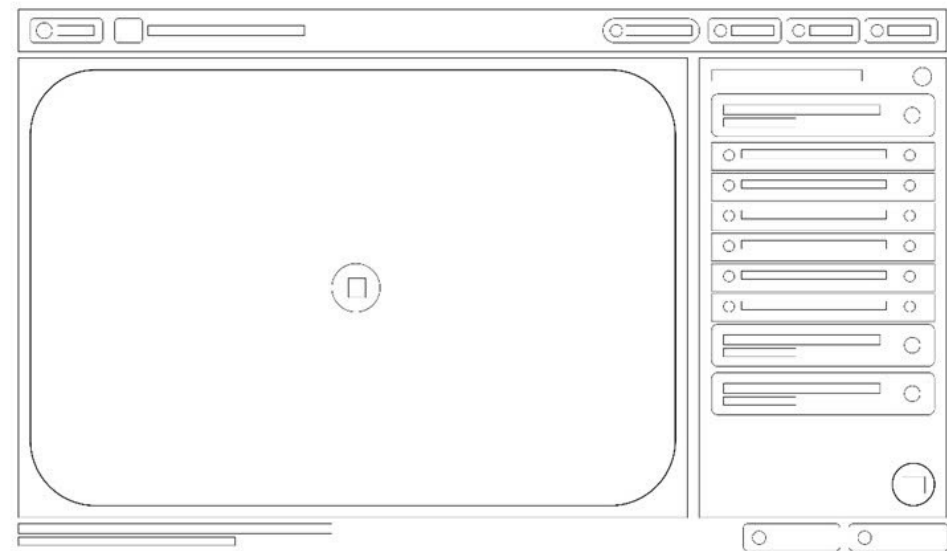


Figure 4.6: Lesson Display UI Wireframe

The learning interface wireframe, as illustrated in Figure 4.x, illustrates the layout of the learning screen when viewing a specific lesson. The main area is used to display learning content, while a side panel lists the lessons within the course, allowing users to easily navigate between lessons during the learning process.

4.2.2 Layer design

This section presents the detailed design of attributes and methods for the core classes in the online learning management system. The system is built following a layered architecture, ensuring separation between data (Entity), business logic (Service), and communication (Controller).

Below is the detailed design of the four most important classes: *User*, *Course*, *Learning*, and *Payment*.

Table 4.6: User Class

Class	User
Attributes	id: UUID username: String email: String password: String mobileNumber: String role: UserRole isActive: Boolean dob: String gender: String location: String profession: String linkedin_url: String github_url: String profileImage: byte[] createdAt: LocalDateTime updatedAt: LocalDateTime lastActiveAt: LocalDateTime
Methods	onCreate(): void onUpdate(): void

Through the *User* class, the system manages and stores user information, and supports authentication, authorization, and user activity tracking.

The *User* class includes the following attributes:

- **id**: the unique identifier of a user in the system.
- **username**: the username of the user.
- **email**: the email address of the user, used for authentication and communica-

tion.

- **password**: the hashed password of the user.
- **mobileNumber**: the phone number of the user.
- **role**: the role of the user in the system (USER, INSTRUCTOR, TA, ADMIN).
- **isActive**: indicates the active status of the user account.
- **dob**: the date of birth of the user.
- **gender**: the gender of the user.
- **location**: the living area information of the user.
- **profession**: the profession of the user.
- **linkedin_url**: the LinkedIn profile URL of the user.
- **github_url**: the GitHub profile URL of the user.
- **profileImage**: the profile avatar image of the user.
- **createdAt**: the account creation time.
- **updatedAt**: the most recent user information update time.
- **lastActiveAt**: the most recent time the user was active on the system.

Methods description of the User class

- **onCreate()**: This method is invoked when creating a new user. It initializes time values such as *createdAt*, *updatedAt*, and *lastActiveAt*.
- **onUpdate()**: This method is invoked when user information is updated. It updates the value of *updatedAt*.

Table 4.7: Course Class

Class	Course
Attributes	course_id: UUID course_name: String price: int instructor: String description: String learningOutcomes: String p_link: String y_link: String tags: String status: CourseStatus plannedDays: Integer totalDurationMinutes: Integer lessonsCount: Integer commentsCount: Integer rating: Double
Methods	computeStatistics (comments: List<Comment>) : void

Through the *Course* class, the system stores course information to support content display, classification, approval status, and the computation of statistics (duration, number of lessons, rating, etc.).

The *Course* class includes the following attributes:

- **course_id**: the unique identifier of a course in the system.
- **course_name**: the course name.
- **price**: the selling price of the course.
- **instructor**: the displayed instructor name for the course.
- **description**: the course description.
- **learningOutcomes**: the outcomes after completing the course (JSON format).
- **p_link**: the PDF document link of the course.
- **y_link**: the YouTube video link of the course.
- **tags**: labels/keywords to support search and classification.

- **status**: the course status (PENDING/APPROVED/REJECTED).
- **plannedDays**: the planned number of study days to complete the course.
- **totalDurationMinutes**: the total duration of the course (minutes), computed dynamically.
- **lessonsCount**: the total number of lessons, computed dynamically.
- **commentsCount**: the total number of valid comments, computed dynamically.
- **rating**: the average rating score of the course, computed dynamically.

Method description of the Course class

- **computeStatistics(comments)**: This method computes display statistics for a course, including *totalDurationMinutes*, *lessonsCount*, *commentsCount*, and *rating*, based on lesson data and the list of comments/ratings.

Table 4.8: Learning Class

Class	Learning
Attributes	id: UUID enrolledAt: LocalDateTime
Methods	onCreate(): void

Through the *Learning* class, the system manages a user's enrollment in a course and stores the enrollment time.

The *Learning* class includes the following attributes:

- **id**: the unique identifier for each course enrollment.
- **enrolledAt**: the time when the user enrolled in the course.

Method description of the Learning class

- **onCreate()**: This method is invoked when creating a new enrollment record. It automatically assigns *enrolledAt* to the current time if it is not set.

Table 4.9: Payment Class

Class	Payment
Attributes	id: UUID amount: long currency: String txnRef: String orderInfo: String orderType: String locale: String bankCode: String ipAddress: String status: PaymentStatus responseCode: String transactionStatus: String transactionNo: String payDate: LocalDateTime createdAt: LocalDateTime updatedAt: LocalDateTime
Methods	onCreate(): void onUpdate(): void

Through the *Payment* class, the system manages payment transaction information (VNPay integration), stores transaction status, and related response information.

The *Payment* class includes the following attributes:

- **id**: the unique identifier of the payment transaction.
- **amount**: the payment amount.
- **currency**: the currency type (e.g., VND).
- **txnRef**: the unique reference code for each transaction.
- **orderInfo**: the order description content.
- **orderType**: the order type based on payment configuration.
- **locale**: the payment interface language.
- **bankCode**: the selected bank code (if any).
- **ipAddress**: the user's IP address during payment.

- **status**: the payment status (PENDING/PAID/FAILED).
- **responseCode**: the response code returned from the payment gateway (VN-Pay).
- **transactionStatus**: the transaction status returned from VNPay.
- **transactionNo**: the transaction number issued by VNPay.
- **payDate**: the successful payment time.
- **createdAt**: the transaction creation time.
- **updatedAt**: the most recent transaction update time.

Method description of the **Payment** class

- **onCreate()**: This method is invoked when creating a new payment transaction. It automatically assigns *createdAt* and *updatedAt* to the current time.
- **onUpdate()**: This method is invoked when updating a transaction (e.g., updating status). It automatically updates *updatedAt*.

Table 4.10: CourseService Class

Method	Return Type
<code>getAllCourses()</code>	<code>List<Course></code>
<code>getAllCoursesFiltered(String categoryId, Boolean free)</code>	<code>List<Course></code>
<code>createCourse(Course course, UUID creatorUserId)</code>	<code>Course</code>
<code>updateCourse(UUID id, Course updatedCourse)</code>	<code>Course</code>
<code>deleteCourse(UUID id)</code>	<code>void</code>
<code>loadCourseStatistics(Course course)</code>	<code>void</code>

Explanation of the **CourseService** class

The *CourseService* class handles business operations related to course management in the system.

- **getAllCourses**: Retrieve all courses, load related data, and compute statistics for each course.
- **getAllCoursesFiltered**: Retrieve approved courses and filter by category or free/paid conditions.
- **createCourse**: Create a new course created by the teacher, set the default status to *PENDING*, and store it in the system.

- **updateCourse:** Update course information by ID, process data changes, and save.
- **deleteCourse:** Delete the course and all related data in the correct dependency order.
- **loadCourseStatistics:** Load modules, lessons, and comments data, and compute display statistics for the course.

Table 4.11: LearningService Class

Method	Return Type
getLearningCourses(UUID userId)	List<Course>
enrollCourse(EnrollRequest enrollRequest)	String
enrollUserInCourse(User user, Course course)	Learning
isUserEnrolled(User user, Course course)	boolean

Explanation of the LearningService class

The *LearningService* class handles business operations related to course enrollment and course participation.

- **getLearningCourses:** Retrieve the list of courses that the user has enrolled in or has been assigned to (TA).
- **enrollCourse:** Perform course enrollment based on the user's request.
- **enrollUserInCourse:** Validate and create the course enrollment record and learning progress for the user.
- **isUserEnrolled:** Check whether the user has the right to learn the course.

Course Enrollment and Payment Sequence Diagram

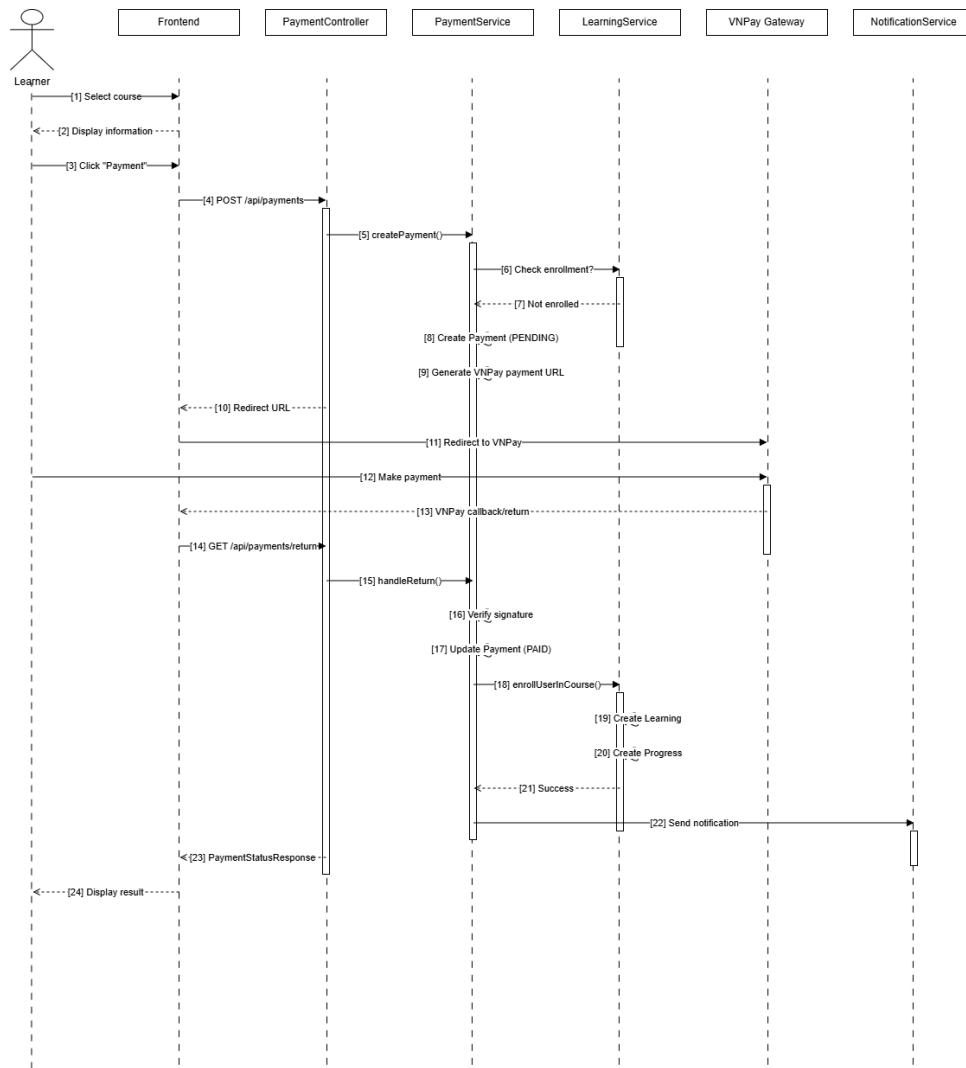


Figure 4.7: Course Enrollment and Payment Sequence Diagram

Lesson Learning and Progress Update Sequence Diagram

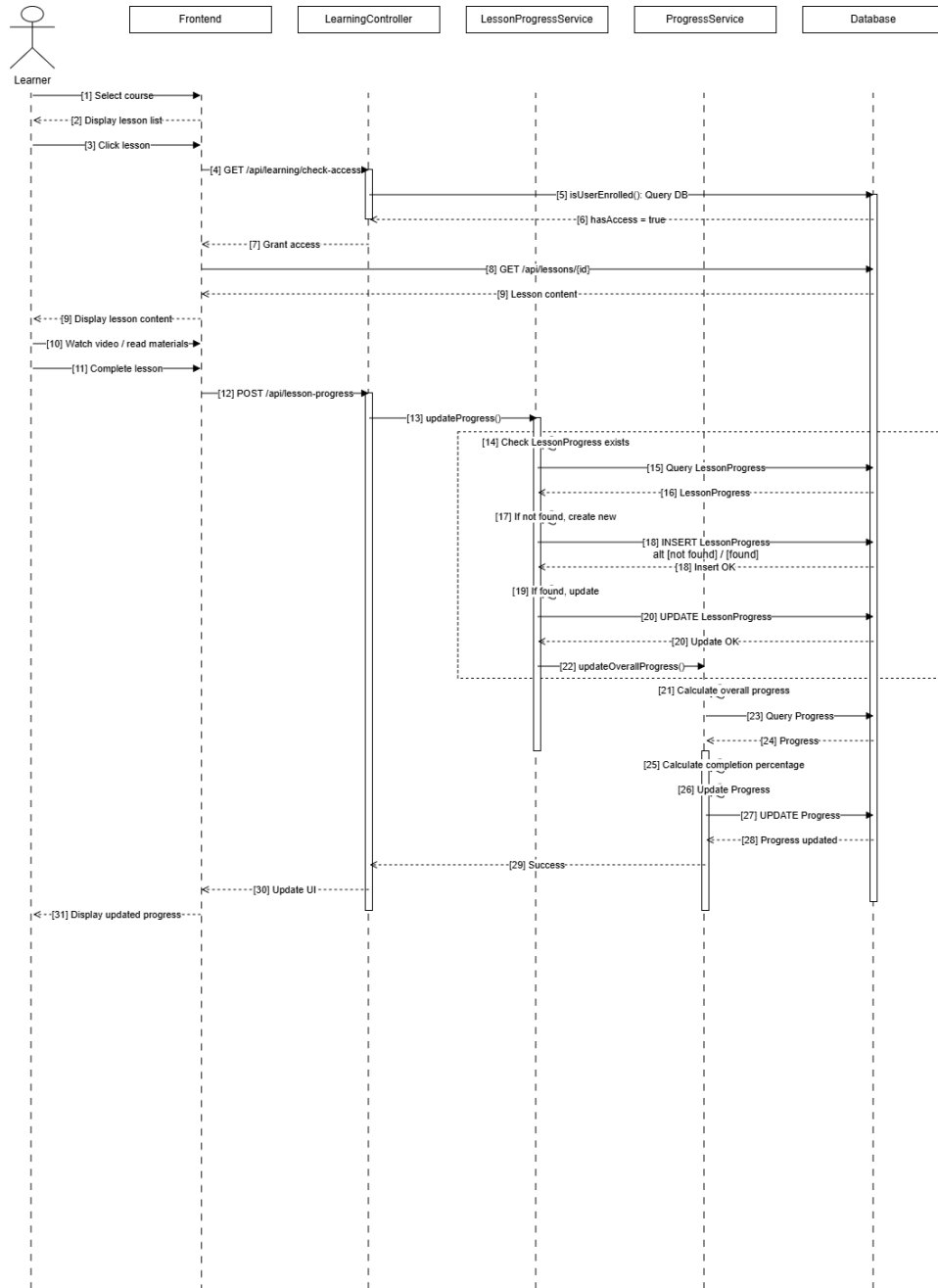


Figure 4.8: Course Creation and Approval Sequence Diagram

Course Creation and Approval Sequence Diagram

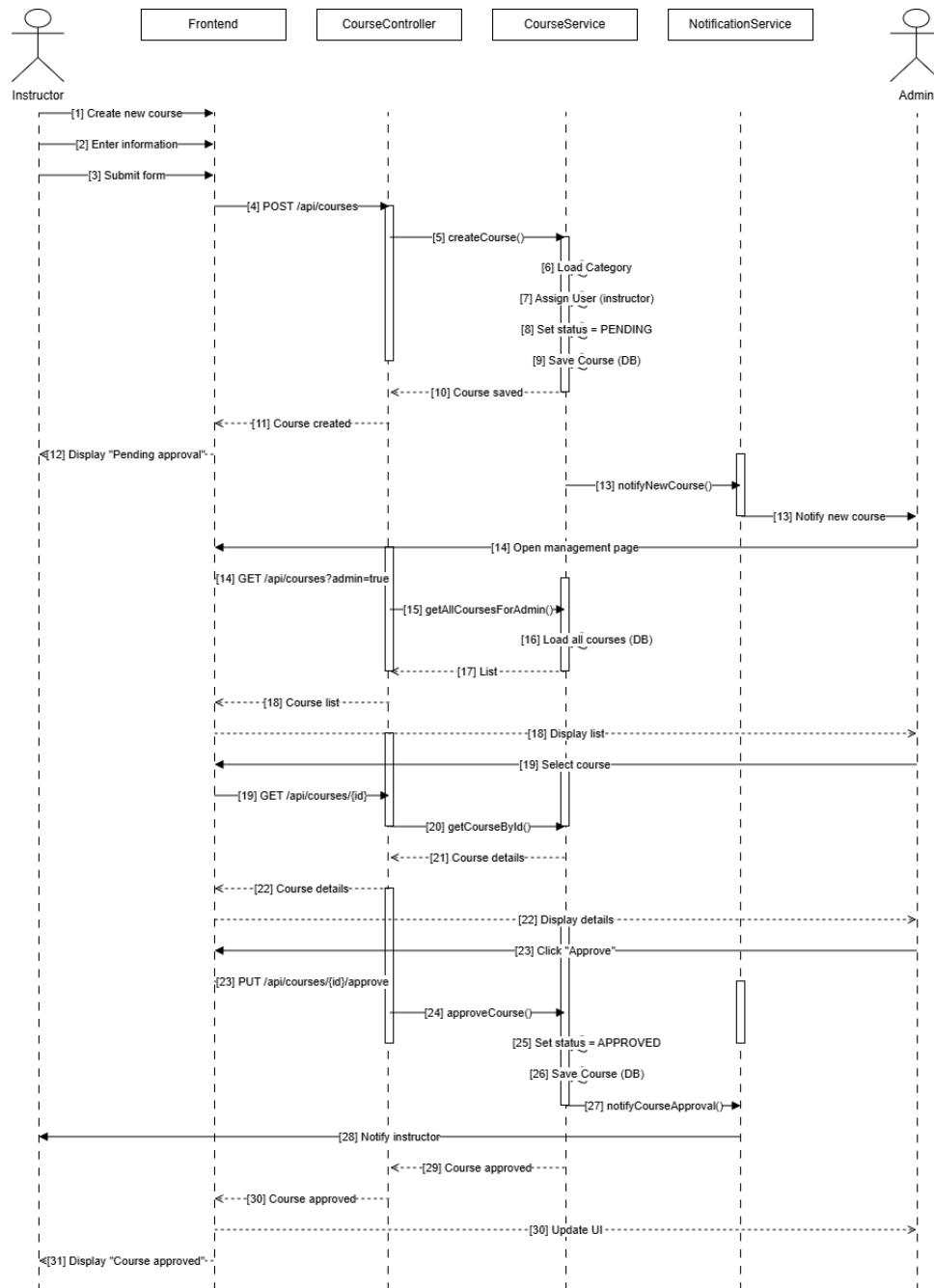


Figure 4.9: Lesson Learning and Progress Update Sequence Diagram

4.2.3 Database design

Entity–Relationship Diagram :

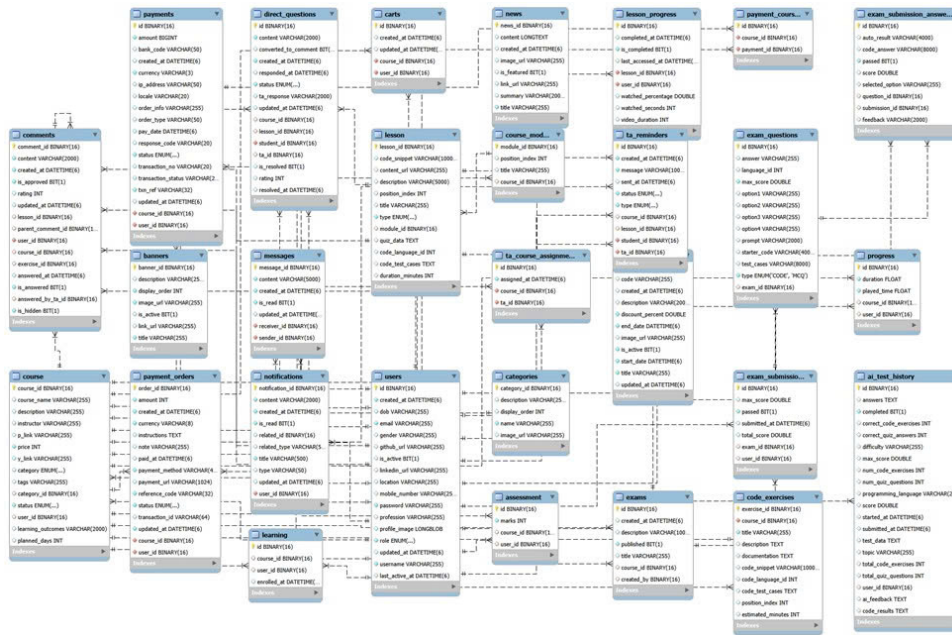


Figure 4.10: Entity-Relationship Diagram (ERD)

The system uses a MySQL relational database for data storage, with tables designed as follows:

Table 4.12: users Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	User primary key (UUID)
2	created_at	DATETIME(6)	Account creation time
3	dob	VARCHAR(255)	Date of birth
4	email	VARCHAR(255)	User email
5	gender	VARCHAR(255)	Gender
6	github_url	VARCHAR(255)	GitHub link
7	is_active	BIT(1)	Account active status
8	linkedin_url	VARCHAR(255)	LinkedIn link
9	location	VARCHAR(255)	Region / residence
10	mobile_number	VARCHAR(255)	Phone number
11	password	VARCHAR(255)	Password (hashed)
12	profession	VARCHAR(255)	Profession
13	profile_image	LOB	Profile avatar image
14	role	ENUM(...)	Role (USER/INSTRUCTOR/-TA/ADMIN)
15	updated_at	DATETIME(6)	Most recent update time
16	username	VARCHAR(255)	Username

No.	Column Name	Data Type	Description
17	last_active_at	DATETIME(6)	Last active time

Table 4.13: categories Table Design

No.	Column Name	Data Type	Description
1	category_id	BINARY(16)	Category primary key
2	description	VARCHAR(255)	Category description
3	display_order	INT	Display order
4	name	VARCHAR(255)	Category name
5	image_url	VARCHAR(255)	Category image

Table 4.14: course Table Design

No.	Column Name	Data Type	Description
1	course_id	BINARY(16)	Course primary key
2	course_name	VARCHAR(255)	Course name
3	description	VARCHAR(255)	Short description
4	instructor	VARCHAR(255)	Displayed teacher name
5	p_link	VARCHAR(255)	Document link (PDF/drive/...)
6	price	INT	Course price
7	y_link	VARCHAR(255)	YouTube video link
8	category	ENUM(...)	Classification field (as defined in ERD)
9	tags	VARCHAR(255)	Search tags
10	category_id	BINARY(16)	FK → categories.category_id
11	status	ENUM(...)	Status (PENDING/APPROVED/REJECTED...)
12	user_id	BINARY(16)	FK → users.id (creator/teacher)
13	learning_outcomes	VARCHAR(2000)	Learning objectives/outcomes
14	planned_days	INT	Planned number of study days

Table 4.15: course_modules Table Design

No.	Column Name	Data Type	Description
1	module_id	BINARY(16)	Module primary key
2	position_index	INT	Module order in the course
3	title	VARCHAR(255)	Module title

No.	Column Name	Data Type	Description
4	course_id	BINARY(16)	FK → course.course_id

Table 4.16: lesson Table Design

No.	Column Name	Data Type	Description
1	lesson_id	BINARY(16)	Lesson primary key
2	code_snippet	VARCHAR(10000)	Sample code snippet (ERD shows as VARCHAR(1000...))
3	content_url	VARCHAR(255)	Content/video/document link
4	description	VARCHAR(5000)	Detailed description
5	position_index	INT	Lesson order within the module
6	title	VARCHAR(255)	Lesson title
7	type	ENUM(...)	Lesson type (VIDEO/QUIZ/-CODE/...)
8	module_id	BINARY(16)	FK → course_modules.module_id
9	quiz_data	TEXT	Quiz data (if any)
10	code_language_id	INT	Programming language identifier
11	code_test_cases	TEXT	Test cases for coding lesson
12	duration_minutes	INT	Lesson duration (minutes)

Table 4.17: learning Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Enrollment primary key
2	course_id	BINARY(16)	FK → course.course_id
3	user_id	BINARY(16)	FK → users.id
4	enrolled_at	DATETIME(6)	Enrollment time

Table 4.18: progress Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Progress primary key
2	duration	FLOAT	Total learning duration (aggregated)
3	played_time	FLOAT	Time already learned/watched
4	course_id	BINARY(16)	FK → course.course_id
5	user_id	BINARY(16)	FK → users.id

Table 4.19: lesson_progress Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Lesson progress primary key
2	completed_at	DATETIME(6)	Completion time
3	is_completed	BIT(1)	Completed or not
4	last_accessed_at	DATETIME(6)	Last accessed time
5	lesson_id	BINARY(16)	FK → lesson.lesson_id
6	user_id	BINARY(16)	FK → users.id
7	watched_percentage	DOUBLE	Watched percentage (video)
8	watched_seconds	INT	Watched seconds
9	video_duration	INT	Total video duration (seconds)

Table 4.20: comments Table Design

No.	Column Name	Data Type	Description
1	comment_id	BINARY(16)	Comment primary key
2	content	VARCHAR(2000)	Comment content
3	created_at	DATETIME(6)	Creation time
4	is_approved	BIT(1)	Approved or not
5	rating	INT	Rating score (if any)
6	updated_at	DATETIME(6)	Update time
7	lesson_id	BINARY(16)	FK → lesson.lesson_id (nullable)
8	parent_comment_id	BINARY(16)	FK → comments.comment_id (reply)
9	user_id	BINARY(16)	FK → users.id
10	course_id	BINARY(16)	FK → course.course_id
11	exercise_id	BINARY(16)	FK → code_exercises.exercise_id (if commenting on code exercise)
12	answered_at	DATETIME(6)	Answered time
13	is_answered	BIT(1)	Answered or not
14	answered_by_ta_id	BINARY(16)	FK → users.id (TA who answered)
15	is_hidden	BIT(1)	Hidden comment

Table 4.21: direct_questions Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Question primary key
2	content	VARCHAR(2000)	Question content
3	converted_to_comment	BIT(1)	Converted to comment or not
4	created_at	DATETIME(6)	Creation time
5	responded_at	DATETIME(6)	Response time
6	status	ENUM(...)	Processing status
7	ta_response	VARCHAR(2000)	Response content from TA
8	updated_at	DATETIME(6)	Update time
9	course_id	BINARY(16)	FK → course.course_id
10	lesson_id	BINARY(16)	FK → lesson.lesson_id
11	student_id	BINARY(16)	FK → users.id (learner)
12	ta_id	BINARY(16)	FK → users.id (TA)
13	is_resolved	BIT(1)	Resolved or not
14	rating	INT	Response rating
15	resolved_at	DATETIME(6)	Resolution completion time

Table 4.22: ta_course_assignments Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Assignment primary key
2	assigned_at	DATETIME(6)	Assignment time
3	course_id	BINARY(16)	FK → course.course_id
4	ta_id	BINARY(16)	FK → users.id (TA)

Table 4.23: ta_reminders Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Reminder primary key
2	created_at	DATETIME(6)	Creation time
3	message	VARCHAR(1000)	Reminder content (ERD shows VARCHAR(10...))
4	sent_at	DATETIME(6)	Sent time
5	status	ENUM(...)	Sending status
6	type	ENUM(...)	Reminder type
7	course_id	BINARY(16)	FK → course.course_id

No.	Column Name	Data Type	Description
8	lesson_id	BINARY(16)	FK → lesson.lesson_id
9	student_id	BINARY(16)	FK → users.id (learner)
10	ta_id	BINARY(16)	FK → users.id (TA)

Table 4.24: payments Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Transaction primary key
2	amount	BIGINT	Payment amount
3	bank_code	VARCHAR(50)	Bank code
4	created_at	DATETIME(6)	Transaction creation time
5	currency	VARCHAR(3)	Currency
6	ip_address	VARCHAR(50)	User IP address
7	locale	VARCHAR(20)	Language
8	order_info	VARCHAR(255)	Order information
9	order_type	VARCHAR(50)	Order type
10	pay_date	DATETIME(6)	Payment time
11	response_code	VARCHAR(20)	Response code (VNPay)
12	status	ENUM(...)	Status (PENDING/PAID/-FAILED...)
13	transaction_no	VARCHAR(20)	Gateway transaction number
14	transaction_status	VARCHAR(20)	Transaction status
15	txn_ref	VARCHAR(32)	Reference code
16	updated_at	DATETIME(6)	Update time
17	course_id	BINARY(16)	FK → course.course_id
18	user_id	BINARY(16)	FK → users.id

Table 4.25: payment_orders Table Design

No.	Column Name	Data Type	Description
1	order_id	BINARY(16)	Payment order primary key
2	amount	INT	Amount
3	created_at	DATETIME(6)	Creation time
4	currency	VARCHAR(8)	Currency
5	instructions	TEXT	Payment instructions
6	note	VARCHAR(255)	Note

No.	Column Name	Data Type	Description
7	paid_at	DATETIME(6)	Successful payment time
8	payment_method	VARCHAR(400)	Payment method
9	payment_url	VARCHAR(1024)	Payment URL
10	reference_code	VARCHAR(32)	Reference code
11	status	ENUM(...)	Order status
12	transaction_id	VARCHAR(64)	Transaction ID
13	updated_at	DATETIME(6)	Most recent update time
14	course_id	BINARY(16)	FK → course.course_id
15	user_id	BINARY(16)	FK → users.id

Table 4.26: payment_courses Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Primary key
2	course_id	BINARY(16)	FK → course.course_id
3	payment_id	BINARY(16)	FK → payments.id

Table 4.27: carts Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Cart primary key
2	created_at	DATETIME(6)	Creation time
3	updated_at	DATETIME(6)	Update time
4	course_id	BINARY(16)	FK → course.course_id
5	user_id	BINARY(16)	FK → users.id

Table 4.28: news Table Design

No.	Column Name	Data Type	Description
1	news_id	BINARY(16)	News primary key
2	content	LONGTEXT	Detailed content
3	created_at	DATETIME(6)	Creation time
4	image_url	VARCHAR(255)	News image
5	is_featured	BIT(1)	Featured news
6	link_url	VARCHAR(255)	Attached link
7	summary	VARCHAR(2000)	Summary

No.	Column Name	Data Type	Description
8	title	VARCHAR(255)	Title

Table 4.29: banners Table Design

No.	Column Name	Data Type	Description
1	banner_id	BINARY(16)	Banner primary key
2	description	VARCHAR(255)	Description
3	display_order	INT	Display order
4	image_url	VARCHAR(255)	Banner image
5	is_active	BIT(1)	Display status
6	link_url	VARCHAR(255)	Link on click
7	title	VARCHAR(255)	Title

Table 4.30: messages Table Design

No.	Column Name	Data Type	Description
1	message_id	BINARY(16)	Message primary key
2	content	VARCHAR(5000)	Content
3	created_at	DATETIME(6)	Creation time
4	is_read	BIT(1)	Read status
5	updated_at	DATETIME(6)	Update time
6	receiver_id	BINARY(16)	FK → users.id (receiver)
7	sender_id	BINARY(16)	FK → users.id (sender)

Table 4.31: notifications Table Design

No.	Column Name	Data Type	Description
1	notification_id	BINARY(16)	Notification primary key
2	content	VARCHAR(2000)	Notification content
3	created_at	DATETIME(6)	Creation time
4	is_read	BIT(1)	Read or not
5	related_id	BINARY(16)	Related object ID
6	related_type	VARCHAR(50)	Related object type
7	title	VARCHAR(500)	Title
8	type	VARCHAR(50)	Notification type
9	updated_at	DATETIME(6)	Most recent update time

No.	Column Name	Data Type	Description
10	user_id	BINARY(16)	FK → users.id

Table 4.32: exams Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Exam primary key
2	created_at	DATETIME(6)	Creation time
3	description	VARCHAR(1000)	Description
4	published	BIT(1)	Publish status
5	title	VARCHAR(255)	Title
6	course_id	BINARY(16)	FK → course.course_id
7	created_by	BINARY(16)	FK → users.id (creator)

Table 4.33: exam_questions Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Question primary key
2	answer	VARCHAR(255)	Correct answer
3	language_id	INT	Language (for CODE question)
4	max_score	DOUBLE	Maximum score
5	option1	VARCHAR(255)	Option 1
6	option2	VARCHAR(255)	Option 2
7	option3	VARCHAR(255)	Option 3
8	option4	VARCHAR(255)	Option 4
9	prompt	VARCHAR(2000)	Question prompt
10	starter_code	VARCHAR(4000)	Starter code
11	test_cases	VARCHAR(8000)	Test cases
12	type	ENUM('CODE', 'MCQ', 'MULTI')	Question type
13	exam_id	BINARY(16)	FK → exams.id

Table 4.34: exam_submissions Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Submission primary key
2	max_score	DOUBLE	Maximum score
3	passed	BIT(1)	Pass/fail

No.	Column Name	Data Type	Description
4	submitted_at	DATETIME(6)	Submission time
5	total_score	DOUBLE	Total score
6	exam_id	BINARY(16)	FK → exams.id
7	user_id	BINARY(16)	FK → users.id

Table 4.35: exam_submission_answers Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Answer primary key
2	auto_result	VARCHAR(4000)	Automatic grading result
3	code_answer	VARCHAR(8000)	Code submission answer
4	passed	BIT(1)	Correct/incorrect or pass/fail
5	score	DOUBLE	Question score
6	selected_option	VARCHAR(255)	Selected option (MCQ)
7	question_id	BINARY(16)	FK → exam_questions.id
8	submission_id	BINARY(16)	FK → exam_submissions.id
9	feedback	VARCHAR(2000)	Feedback/comments

Table 4.36: code_exercises Table Design

No.	Column Name	Data Type	Description
1	exercise_id	BINARY(16)	Coding exercise primary key
2	course_id	BINARY(16)	FK → course.course_id
3	title	VARCHAR(255)	Title
4	description	TEXT	Description
5	documentation	TEXT	Documentation/guidelines
6	code_snippet	VARCHAR(10000)	Sample code (ERD shows VARCHAR(1000...))
7	code_language_id	INT	Programming language identifier
8	code_test_cases	TEXT	Test cases
9	position_index	INT	Exercise order
10	estimated_minutes	INT	Estimated time (minutes)

Table 4.37: assessment Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	Assessment primary key
2	marks	INT	Score/rating
3	course_id	BINARY(16)	FK → course.course_id
4	user_id	BINARY(16)	FK → users.id

Table 4.38: ai_test_history Table Design

No.	Column Name	Data Type	Description
1	id	BINARY(16)	AI test history primary key
2	answers	TEXT	Answer list
3	completed	BIT(1)	Completed or not
4	correct_code_exercises	INT	Number of correct coding exercises
5	correct_quiz_answers	INT	Number of correct quiz answers
6	difficulty	VARCHAR(255)	Difficulty level
7	max_score	DOUBLE	Maximum score
8	num_code_exercises	INT	Number of coding exercises in the test
9	num_quiz_questions	INT	Number of quiz questions in the test
10	programming_language	VARCHAR(255)	Programming language
11	score	DOUBLE	Achieved score
12	started_at	DATETIME(6)	Start time
13	submitted_at	DATETIME(6)	Submission time
14	test_data	TEXT	Test/exam data
15	topic	VARCHAR(255)	Topic
16	total_code_exercises	INT	Total coding exercises
17	total_quiz_questions	INT	Total quiz questions
18	user_id	BINARY(16)	FK → users.id
19	ai_feedback	TEXT	Feedback from AI
20	code_results	TEXT	Code grading results

4.3 Application Building

4.3.1 Libraries and Tools

A summarized list of the libraries and tools used is presented in Table 4.39.

Table 4.39: List of Libraries and Tools Used (Summarized)

No.	Purpose	Tool / Library	Version	URL
1	Frontend IDE	Visual Studio Code	(machine-dependent)	https://code.visualstudio.com/
2	Backend IDE	IntelliJ IDEA	(machine-dependent)	https://www.jetbrains.com/idea/
3	Frontend Language	JavaScript (ES6+)	–	https://developer.mozilla.org/en-US/docs/Web/JavaScript
4	Frontend Framework	React	19.1.1	https://react.dev/
5	Frontend Build Tool	Vite	7.1.12	https://vitejs.dev/
6	UI & CSS	Ant Design, Tailwind CSS	5.28.0 / 4.1.16	https://ant.design/ ; https://tailwindcss.com/
7	Routing	React Router DOM	7.9.5	https://reactrouter.com/
8	HTTP Client	Axios	1.13.1	https://axios-http.com/
9	Backend Language	Java	23	https://www.oracle.com/java/
10	Backend Framework	Spring Boot	3.5.6	https://spring.io/projects/spring-boot
11	ORM & Security	Spring Data JPA, Spring Security	–	https://spring.io/
12	JWT	JWT	0.11.5	https://github.com/jwtkt/jjwt
13	Backend Build Tool	Gradle	–	https://gradle.org/
14	Database	MySQL	–	https://www.mysql.com/
15	Code Generation	Lombok	–	https://projectlombok.org/

No.	Purpose	Tool / Library	Version	URL
16	Code Execution API	Judge0 CE	–	https://ce.judge0.com/
17	AI Integration	Google Gemini API	2.5 Flash	https://ai.google.dev/
18	Payment Gateway	VNPay	Sandbox	https://sandbox.vnpayment.vn/

4.3.2 Achievement

The table below summarizes the main deployment metrics of the system.

Table 4.40: Detailed Application Information Statistics

Group	Metric	Result
Frontend Web	Lines of code	~15,000 lines
	Number of JSX files	65 files
	Number of JavaScript files	28 files
	Number of components	93 components
	Number of npm libraries	27 packages
	Project size	~50 MB (including node_modules)
	Build size	~5 MB
Backend API	Lines of code	~25,000 lines
	Number of Java files	156 files
	Number of Controllers	28
	Number of Services	29
	Number of Entities	25
	Number of Repositories	25
	Number of DTOs	31
	Number of Gradle libraries	15 packages
	Project size	~100 MB (including dependencies)
Database	DBMS	MySQL
	Number of tables	25+ tables
	Migration scripts	8 files
Overview	Total lines of code	~40,000 lines
	Total source files	249+ files
	Primary languages	Java, JavaScript
	Primary frameworks	Spring Boot, React
	Architecture	RESTful API, SPA

4.3.3 Illustration of main functions

To demonstrate the results achieved during the system development process, this section presents detailed descriptions of the interface screens corresponding to the core functional groups of the application. The introduced interfaces include: the home page interface, the course list interface, the course detail interface, the learning interface (including a video viewer, quizzes, learning materials, and a source code editor), the admin dashboard interface, the course management interface, the user information interface, and the learner progress management interface.

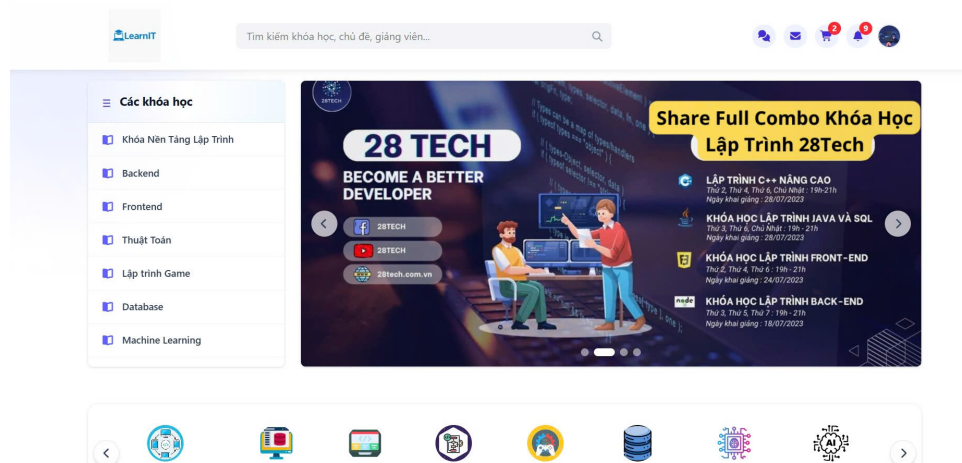


Figure 4.11: Home Page Interface

The home page interface, as illustrated in Figure 4.x, is the first screen that users encounter when accessing the system. The interface is designed intuitively, featuring a search bar and functional icons at the top, enabling users to quickly search for information and manage personal data. The left navigation panel displays the main course categories, supporting quick access to different learning domains. The central area presents banners introducing featured courses along with training fields, allowing users to easily explore and select suitable content. Overall, the home page interface serves as the primary navigation hub, contributing to an enhanced user experience when using the system.

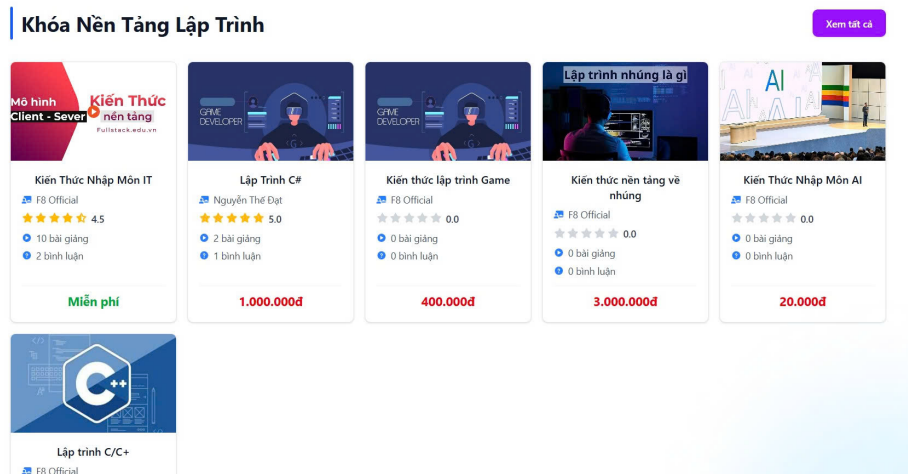


Figure 4.12: Interface displaying featured courses

The interface displaying featured courses, as illustrated in Figure 4.x, presents a list of courses in the programming fundamentals category in the form of course cards. Each card shows basic course information, including a representative image, course title, instructor, number of lectures, number of questions, and price. Through this interface, users can quickly obtain an overview of the courses and select those that best suit their learning needs.

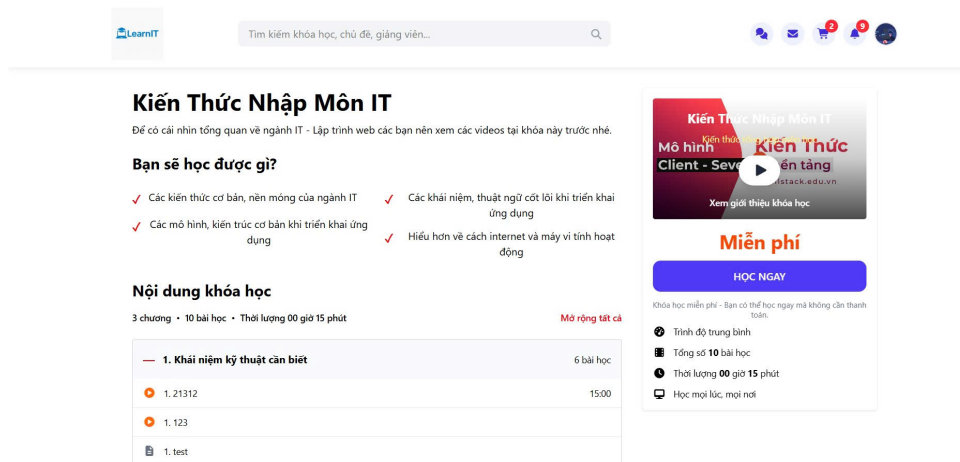


Figure 4.13: The Course Detail Interface

The course detail interface, as illustrated in Figure 4.x, provides comprehensive information related to a specific course. In this interface, users can view an overview of the course through the course title, a brief description, and an introductory video. The main course content is presented below in the form of a list of chapters and lessons, along with the number of lectures and the learning duration. On the right side of the interface, the system displays general information such as the course level, total number of lessons, total duration, and whether the course

is free or paid, while also providing action buttons that allow users to enroll in or start the course. The course detail interface enables users to easily understand the course content and conveniently decide to participate in the course.

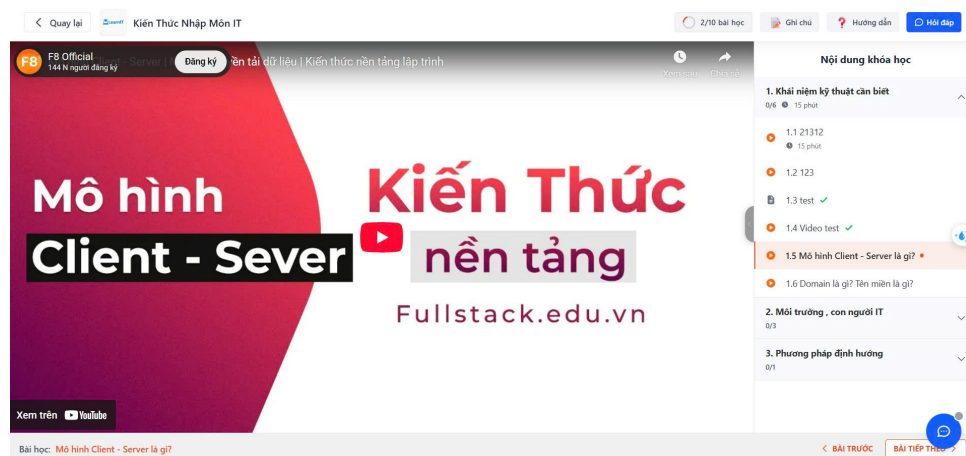


Figure 4.14: Entity–Relationship Diagram (ERD)

The video-based learning interface, as illustrated in Figure 4.x, is designed to support users during the learning process through video lectures. The main area of the interface displays the video player, allowing users to watch instructional content with basic playback controls. On the right side, the system presents the course content in a structured list of chapters and lessons, enabling learners to easily track their progress and navigate between lessons. Additional functional buttons such as notes, guidance, and question-and-answer support are provided to enhance interaction and learning effectiveness. This interface facilitates a focused and flexible learning experience, helping users efficiently follow and complete video-based lessons.

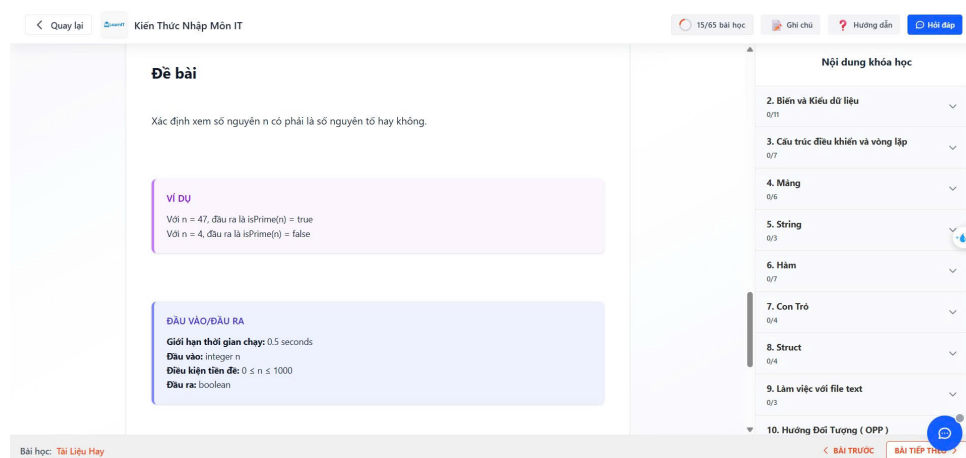


Figure 4.15: The Learning Material Interface

The learning material interface, as illustrated in Figure 4.x, displays textual con-

tent and illustrative examples, supporting learners in reviewing and consolidating theoretical knowledge.



Figure 4.16: The Quiz Interface

The quiz interface, as illustrated in Figure 4.x, presents multiple-choice questions to help learners review and reinforce their knowledge after each lesson, providing immediate feedback on their answers.

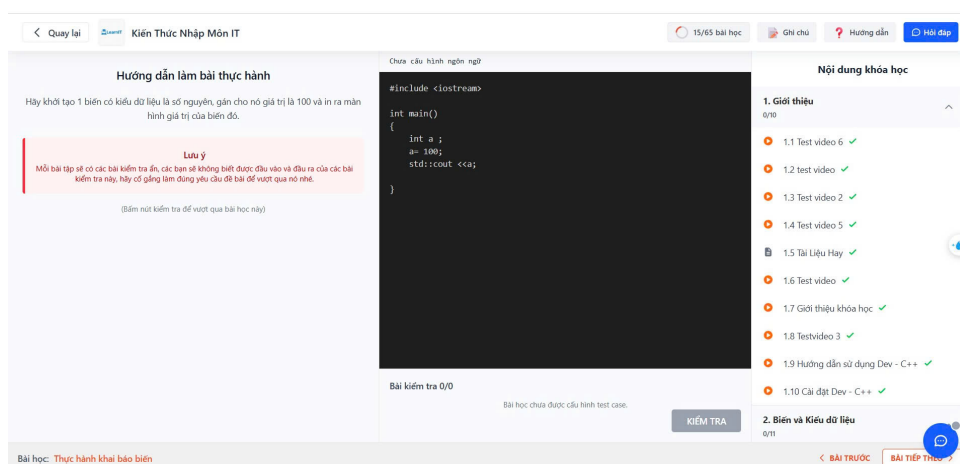


Figure 4.17: The Coding Practice Interface

The coding practice interface, as illustrated in Figure 4.x, allows learners to view problem descriptions and write source code directly within the system to complete practical exercises.

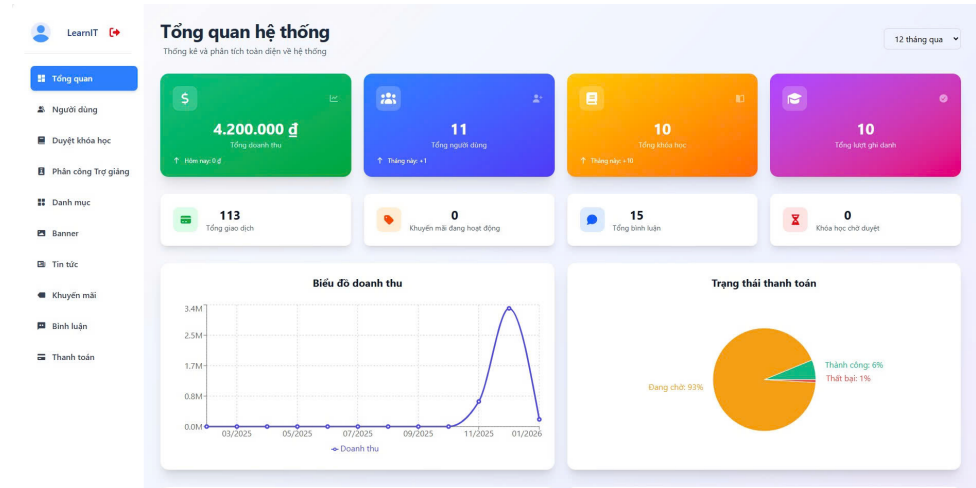


Figure 4.18: The Course Management Interface

The course management interface, as illustrated in Figure 4.x, allows administrators to create, view, and manage courses within the system. The interface displays a list of courses along with essential information such as course title, instructor, price, status, and number of enrolled learners. In addition, functional buttons are provided to manage learners, examinations, learning materials, and coding exercises for each course, supporting efficient course administration.

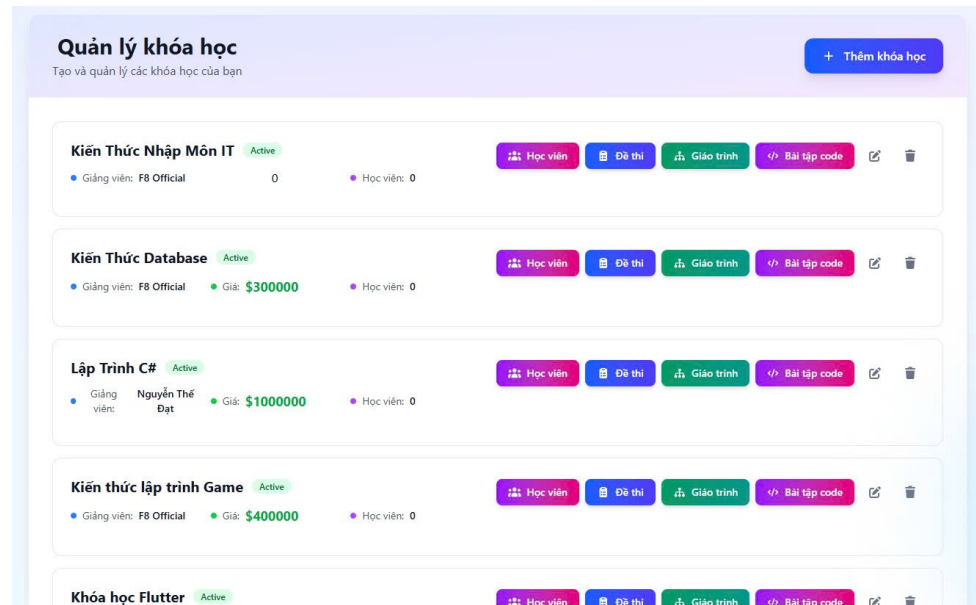


Figure 4.19: The Admin Dashboard Interface

The admin dashboard interface, as illustrated in Figure 4.x, provides an overview of the system through statistical summaries and visual analytics. The interface displays key indicators such as total revenue, number of users, total courses, and enrollments, allowing administrators to quickly monitor system performance. In addition, charts and graphs are used to visualize revenue trends and payment status,

supporting effective management and decision-making.

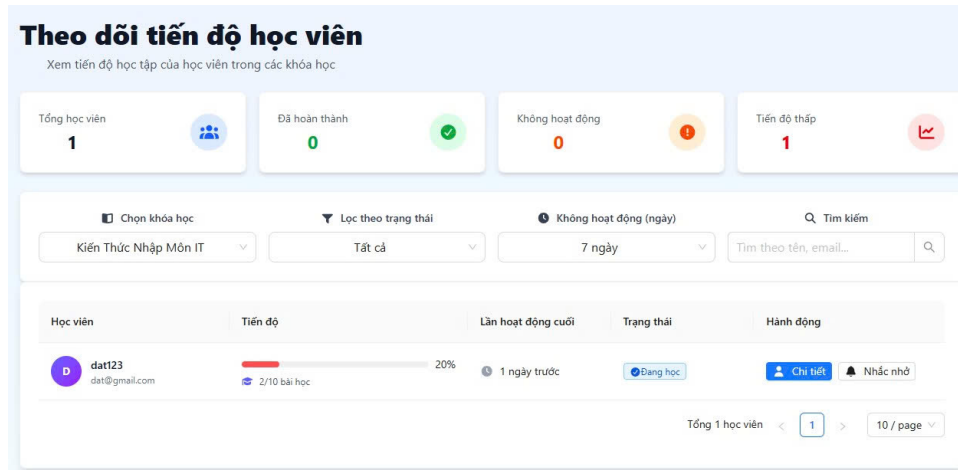
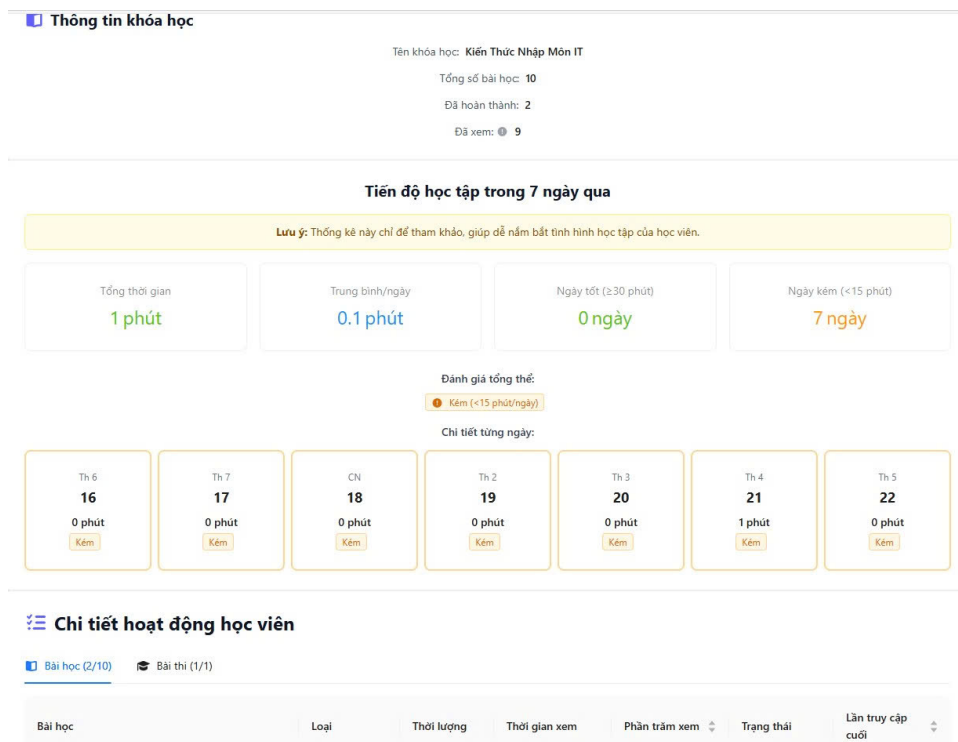


Figure 4.20: The Learner Progress Tracking Interface

The learner progress tracking interface, as illustrated in Figure 4.x, enables administrators to monitor learners' learning progress across courses. The interface provides summary statistics such as total learners, completion status, inactivity, and low progress levels. In addition, filtering and search functions are supported to help administrators easily track individual learner progress and take appropriate actions when necessary.



Chi tiết hoạt động học viên

Bài học (2/10) Bài thi (1/1)

Bài học	Loại	Thời lượng	Thời gian xem	Phần trăm xem	Trạng thái	Lần truy cập cuối
21312 Khái niệm kỹ thuật cần biết	VIDEO	15 phút	0:00 0	0%	Chưa bắt đầu	9 ngày trước
123 Khái niệm kỹ thuật cần biết	VIDEO	N/A	0:00	0%	Chưa bắt đầu	5 ngày trước
test Khái niệm kỹ thuật cần biết	MATERIAL	N/A	0:00	100%	Hoàn thành	4 ngày trước
Video test Khái niệm kỹ thuật cần biết	VIDEO	N/A	1:49	90%	Hoàn thành	1 ngày trước
Mô hình Client - Server là gì? Khái niệm kỹ thuật cần biết	VIDEO	N/A	0:00	0%	Chưa bắt đầu	4 ngày trước
Domain là gì? Tên miền là gì? Khái niệm kỹ thuật cần biết	VIDEO	N/A	0:00	0%	Chưa bắt đầu	4 ngày trước
Học IT cần tố chất gì? Góc nhìn khác từ chuyên gia định hướng giáo dục Môi trường , con người IT	VIDEO	N/A	0:00	0%	Chưa bắt đầu	4 ngày trước
Sinh viên IT đi thực tập tại doanh nghiệp cần biết những gì? Môi trường , con người IT	VIDEO	N/A	0:00	0%	Chưa bắt đầu	5 ngày trước
Trải nghiệm thực tế sau 2 tháng làm việc tại doanh nghiệp của học viên F8? Môi trường , con người IT	VIDEO	N/A	0:00	0%	Chưa bắt đầu	5 ngày trước

Figure 4.21: The Detailed Learner Progress Interface

The detailed learner progress interface, as illustrated in Figure 4.x, provides in-depth information about an individual learner's learning activities within a specific course. The interface displays lesson-level details such as content type, viewing duration, completion percentage, learning status, and last access time. In addition, summary statistics and learning activity over a recent period are presented to help administrators evaluate learning behavior and identify potential learning issues.

4.4 Testing

Testing is an important step in the software development process to ensure that business workflows operate as expected and that data is stored correctly. To ensure system quality, the student used the Black-box Testing technique to test the main functions of the system.

Black-box Testing is used to test functions without knowing the internal structure of the system in detail. The student focused on testing the inputs and outputs of the functions, ensuring that the system works correctly under real-world usage scenarios.

The student designed and executed tests for the three most important functions of the system:

1. Register / Login – User authentication function
2. Pay for Courses – Payment transaction processing function

3. Take Coding Exercises – Code execution and evaluation function

4.4.1 Testing the Register / Login Function

The Register / Login function is the most basic and important function of the system, ensuring security and access control. The system supports login with three roles: Learner, Teacher, and Teaching Assistant.

Table 4.41: Register / Login Function Testing

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
1	Email does not exist in the system	Enter a non-existing email and any password, select a role, click “Login”	Error message: “Incorrect email or password”	Error message: “Incorrect email or password”	Pass
2	Incorrect password	Enter a valid email but an incorrect password, select a role, click “Login”	Error message: “Incorrect email or password”	Error message: “Incorrect email or password”	Pass
3	Role not selected	Enter valid email and password but do not select a role, click “Login”	Error message: “Please select a login role”	Error message: “Please select a login role”	Pass
4	Role does not match the account	Enter valid email and password but choose the wrong role (e.g., the account is Learner but select Teacher), click “Login”	Error message: “This account does not have the [role] permission. Please select the correct role.”	Error message: “This account does not have the [role] permission. Please select the correct role.”	Pass
5	JWT token expired	Login successfully, wait for the token to expire (or delete the token), perform an action that requires authentication	The system requires login again or shows “Session has expired”	The system requires login again or shows “Session has expired”	Pass

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
6	Successful login with Learner role	Enter valid email and password of a Learner account, select role “Learner”, click “Login”	Login succeeds, redirect to learner homepage (/home), save token to localStorage	Login succeeds, redirect to learner homepage (/home), token is saved to localStorage	Pass
7	Successful login with Teacher role	Enter valid email and password of a Teacher account, select role “Teacher”, click “Login”	Login succeeds, redirect to teacher homepage (/teacher-home)	Login succeeds, redirect to teacher homepage (/teacher-home)	Pass
8	Successful login with Teaching Assistant role	Enter valid email and password of a Teaching Assistant account, select role “Teaching Assistant”, click “Login”	Login succeeds, redirect to teaching assistant homepage (/teaching-assistant-home)	Login succeeds, redirect to teaching assistant homepage (/teaching-assistant-home)	Pass
9	Email or password is empty	Leave email or password empty, click “Login”	Validation error message requiring complete information	Validation error message requiring complete information	Pass
10	Invalid email format	Enter an invalid email format (e.g., “test@”, “test.com”), enter password, click “Login”	Validation error message for invalid email format	Validation error message for invalid email format	Pass

The Register / Login function has 10 test cases, including 6 invalid cases and 4 valid cases. The system produced the correct results matching the expected outcomes in every test case. All test cases passed, demonstrating that the Register / Login function operates stably and correctly handles both error cases and successful cases.

4.4.2 Testing the Pay for Courses Function

The Pay for Courses function is important because it is related to financial transactions and access rights to learning content. The system integrates with the VNPay payment gateway to process online payment transactions.

Table 4.42: Pay for Courses Function Testing

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
1	Course already purchased	Select a course that was purchased before, click “Pay” or “Buy course”	Message: “You already own this course” or redirect to learning page	Message: “You already own this course” or redirect to learning page	Pass
2	User not logged in	Not logged in, select a course and click “Buy course”	Require login before payment	Require login before payment	Pass
3	JWT token expires during payment	Login, select a course, the token expires when calling the payment API	Error: “Session has expired”, require login again	Error: “Session has expired”, require login again	Pass
4	Course does not exist	Select a course with a non-existing ID, click “Pay”	Error: “Course does not exist”	Error: “Course does not exist”	Pass
5	Successful VNPay payment	Login, select a not-yet-purchased course, click “Pay”, choose bank, complete payment on VNPay	Redirect to payment result page with status “Success”, course is unlocked, cart is cleared (if paying from cart)	Redirect to payment result page with status “Success”, course is unlocked, cart is cleared	Pass
6	User cancels payment on VNPay	Login, select a course, click “Pay”, go to VNPay but cancel the transaction	Redirect to payment result page with status “Failed” or return to course page	Redirect to payment result page with status “Failed”	Pass

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
7	Payment timeout	Login, select a course, click “Pay”, do not complete payment within the specified time (15 minutes)	Transaction expires, message “Transaction has expired”	Transaction expires, message “Transaction has expired”	Pass
8	Pay from cart (multiple courses)	Login, add multiple courses to cart, click “Checkout”, complete payment	All courses in cart are unlocked, cart is cleared after successful payment	All courses in cart are unlocked, cart is cleared	Pass
9	VNPay connection error	Login, select a course, click “Pay”, but cannot connect to VNPay	Error: “Cannot connect to the payment gateway. Please try again later.”	Error: “Cannot connect to the payment gateway. Please try again later.”	Pass
10	VNPay signature verification fails	Login, select a course, click “Pay”, VNPay returns but the signature is invalid	Error: “Invalid transaction” or reject the transaction	Error: “Invalid transaction”	Pass

The Pay for Courses function has 10 test cases, including 6 invalid cases and 4 valid cases. The system produced the correct results matching the expected outcomes in every test case. All test cases passed, ensuring the safety and reliability of the payment process.

4.4.3 Testing the Take Coding Exercises Function

The Take Coding Exercises function allows learners to write, run, and submit their code. The system integrates with the Judge0 API to execute code and evaluate results. Learners can run their code before submission and view test case results.

Table 4.43: Table 4.5: Take Coding Exercises Function Testing

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
1	User not logged in	Not logged in, access the coding exercise page	Require login or redirect to login page	Require login or redirect to login page	Pass

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
2	Exercise does not exist	Login, access the exercise URL with a non-existing ID	Error: “Exercise not found” and redirect to previous page	Error: “Exercise not found” and redirect to previous page	Pass
3	Learner has not purchased the course	Login with an account that has not purchased the course, access a coding exercise of that course	Message: “You need to purchase the course to access this exercise” or redirect to the course page	Message: “You need to purchase the course to access this exercise”	Pass
4	Code has syntax errors	Login, open an exercise, enter code with syntax errors, click “Run”	Display syntax error from Judge0 (e.g., “SyntaxError”, “Compilation Error”)	Display syntax error from Judge0	Pass
5	Code runs but fails test cases	Login, open an exercise, enter runnable code but output does not match expected output, click “Run”	Show result “Failed” with input, expected output, actual output	Show result “Failed” with detailed information	Pass
6	Code passes some test cases	Login, open an exercise, enter code that passes some test cases but fails others, click “Run”	Show number of passed/failed test cases, highlight passed and failed test cases	Show number of passed/failed test cases, highlight passed and failed test cases	Pass
7	Code passes all test cases	Login, open an exercise, enter code that passes all test cases, click “Submit”	Show “Submission successful”, update exercise completion status, show confetti animation	Show “Submission successful”, update completion status, show confetti	Pass
8	Run code with custom input	Login, open an exercise, enter code, enter custom input in stdin box, click “Run”	Show the output of the code with custom input	Show the output of the code with custom input	Pass

No.	Test Data	Test Procedure	Expected Result	Actual Result	Status
9	Judge0 API connection error	Login, open an exercise, enter code, click “Run” but Judge0 API does not respond	Error: “Cannot connect to the code execution service. Please try again later.”	Error: “Cannot connect to the code execution service. Please try again later.”	Pass
10	Code time-out (runs too long)	Login, open an exercise, enter code with an infinite loop or very long runtime, click “Run”	Message “Time Limit Exceeded” or “Code execution exceeded the time limit”	“Time Limit Exceeded” from Judge0	Pass
11	Programming language does not match the exercise	Login, open an exercise that requires Java, but try to submit Python code	The system only allows code in the specified language of the exercise	The system only allows code in the specified language of the exercise	Pass
12	Submit without running first	Login, open an exercise, enter code but do not run it, click “Submit”	Allow submission or warn “You have not run the code. Are you sure you want to submit?”	Allow direct submission	Pass

The Take Coding Exercises function has 12 test cases, including 5 invalid cases and 7 valid cases. The system produced the correct results matching the expected outcomes in every test case. All test cases passed, ensuring that the Take Coding Exercises function operates stably and integrates well with the Judge0 API.

All important functions were tested thoroughly and are ready for deployment in a real-world environment.

4.5 Deployment

The online programming management and teaching system is built following a Three-tier Architecture and is deployed experimentally in a Development Environment, with an orientation to expand to cloud platforms in the future. Specifically, the Web application (Frontend) is developed using ReactJS and deployed on the Vercel platform to optimize content delivery (CDN). The application server (Backend API) uses Spring Boot and is configured to run on a virtual server environment (such as AWS EC2) or on a local server during the testing phase. The MySQL database management system is used for centralized data storage.

The experimental deployment configuration includes the Backend server running on Java Runtime Environment (JRE) 23 and the Frontend running on a Node.js environment.

Table 4.44: Backend and Frontend (Web) Server Information

No.	Server	Access Address / Domain	Configuration / Notes
1	Backend API Server	http://localhost:8080	Spring Boot 3.5.6, Java 23, Tomcat Server
2	Frontend (Web App)	http://localhost:5173	React 19, Vite, Vercel (Production Build)

Regarding the experimental deployment results, the system was tested on a personal computer environment (CPU Core i5, RAM 8GB) with approximately 15–20 simulated users. The results recorded that the system operated stably, with an average API response time under 800ms for normal tasks and under 1s for code-processing tasks (Judge0). The initial load capacity meets the requirements well for a small classroom scale.

Detailed information about the database server storing the system is as follows:

Table 4.45: Database Server Information

No.	Parameter Name	Value
1	Server Address (Host)	localhost (or AWS RDS Endpoint)
2	DBMS	MySQL Server
3	Version	8.0
4	Connection Port (Port)	3306
5	Charset	utf8mb4 (Supports Vietnamese Unicode)
6	Storage Capacity	Minimum 10GB (Recommended for Production)

In Chapter 4, I presented in detail the design and implementation of the system. The content includes the overall architecture, detailed design of software packages (Frontend/Backend), database specification, and user interface. At the same time, the chapter also describes the practical deployment plan, server configuration, and preliminary testing results, demonstrating the feasibility of the selected technology solution. This is an important basis for evaluating the effectiveness of the thesis in the next chapter.

CHAPTER 5. SOLUTION AND CONTRIBUTION

During the research and development of the system “Online Programming Management and Teaching”, in addition to completing the basic functions of an LMS system, the student focused on solving in-depth technical problems that are specific to programming education such as: secure source code execution, automatic grading, intelligent learning support using AI, and financial transaction security.

This chapter presents in detail the 06 most outstanding solutions and contributions of the thesis. Each section goes from analyzing the practical problem, proposing a specific technical solution, and evaluating the achieved results.

5.1 Secure and High-Performance Automatic Code Evaluation Architecture

5.1.1 Problem Statement

Unlike conventional learning systems that only process text or multiple-choice quizzes, a programming teaching system requires the “Code Execution” feature. This is the most challenging problem with three major issues:

- **System security (Security):** If users are allowed to run arbitrary code on the server, attackers may execute destructive commands (such as `rm -rf /`, fork bomb) or gain unauthorized access to system resources.
- **Language diversity:** The system must support compiling multiple languages (C++, Java, Python, etc.) with different compiler versions without complicating the deployment environment.
- **Performance:** Compiling and running code consumes significant CPU/RAM. If handled synchronously on the same application server, the system will freeze when many users submit at the same time.

5.1.2 Proposed Solution

To thoroughly address this problem, the student designed and implemented a Decoupled Architecture integrated with the Judge0 API in a Docker Sandboxing environment.

Details of the submission processing sequence diagram have been presented in Section 4.2.2.5.

The specific solution includes the following technical components:

- **Docker Sandboxing (Environment isolation):** Each code execution request is executed in a separate Docker container. This container is resource-limited (CPU quota, Memory limit) and has no network access (Network disable),

ensuring that user code cannot affect the physical server (Host OS).

- **Asynchronous Processing mechanism:**
 - The Spring Boot backend does not execute code directly but sends a request (Submission) to the Judge0 Service.
 - Judge0 processes in a queue (Queue) and returns results via Webhook or Polling.
 - This frees the Web Server thread, allowing the system to accept thousands of concurrent requests without bottlenecks.
- **Judge0 CE (Community Edition) integration:** Instead of building compilers from scratch, the system integrates Judge0 — a powerful open-source judge system — supporting more than 50 programming languages immediately with high reliability.

5.1.3 Achieved Results

- **Absolute security:** Successfully prevented 100% of malicious-code attack scenarios during testing.
- **High performance:** The system can handle multiple submissions concurrently without affecting the web browsing experience of other users. The average response time for a simple submission is under 2 seconds.
- **Scalability:** New languages can be added or compiler versions upgraded simply by updating Docker image configurations without modifying system code.

5.2 Programming Virtual Assistant Integrated with Google Gemini AI

5.2.1 Problem Statement

In the online self-learning model, learners often face difficulties when their code contains bugs and there is no instructor beside them to explain immediately. Compiler error messages are often dry and difficult to understand for beginners (for example: Segmentation fault or NullPointerException), leading learners to become discouraged and give up.

5.2.2 Proposed Solution

The student researched and integrated the large language model (LLM) Google Gemini Pro to build the “AI Debugging Assistant” feature.

The solution includes:

- **Prompt Engineering:** Building specialized System Prompts to guide Gemini to act as a “patient programming instructor”. The AI not only provides fixed

code but must explain why the error occurs and suggest solution directions.

- **Prompt structure:** [Role: Programming Tutor] + [Context: User is solving problem X] + [Input: Code & Error Log] + [Constraint: Provide explanation, not just solution].
- **Context-Aware Analysis:** The system sends the problem statement context and the learner's current code so the AI has a comprehensive view, thereby giving more accurate advice.
- **Integration Layer:** Building a Service Wrapper in Spring Boot to communicate with the Google AI API, handling Rate Limiting issues and reformatting responses (Markdown) before displaying them to users.

5.2.3 Achieved Results

- **Instant feedback:** Learners receive detailed explanations of errors within 3–5 seconds.
- **Improved learning experience:** Helps reduce the workload of instructors and teaching assistants in answering basic questions.
- **Personalization:** The AI provides debugging suggestions based on the learner's own coding style, making learning more intuitive.

5.3 Teaching Assistant (TA) Management System and Advanced Authorization

5.3.1 Problem Statement

Open-source LMS systems often only have rigid roles such as Admin, Teacher, and Student. However, in universities or programming training centers, the Teaching Assistant (TA) role is extremely important. A TA needs higher privileges than a learner (to grade assignments, view class lists) but lower privileges than an instructor (cannot delete courses, cannot view revenue). If this permission management is not designed well, it may lead to security risks (Privilege Escalation) or operational difficulties.

5.3.2 Proposed Solution

The student designed a deeply customized Role-Based Access Control (RBAC) system using Spring Security, focusing on solving the authorization problem for TAs.

Technical solution:

- **Dynamic Permission Evaluation:** Instead of checking only a role (ROLE_TA), the system checks the ownership relationship (Assignment). A TA can only

access a course that they are “Assigned” to.

- **Mechanism:** Using annotation `@PreAuthorize("@securityService.isTaAssigned(#courseId) ")` at the Controller layer.
- **Relational data model:** Designing the `TACourseAssignment` table to store the N–N relationship between User (TA) and Course, along with specific permissions (for example: `can_grade`, `can_reply_discussion`).
- **Notification system:** Automatically sending notifications to TAs when there is a new assignment to grade or a new question in discussions, ensuring TAs do not miss tasks.

5.3.3 Achieved Results

- Successfully built a complete business workflow for TAs: from being invited by an instructor into a course, task assignment, to grading and supporting learners.
- Ensured the “Least Privilege” principle: TAs can only operate on authorized resources, ensuring data security for the course.

5.4 Dynamic Statistics Computation Mechanism and Data Consistency

5.4.1 Problem Statement

In a learning system, indicators such as “Average rating” (Rating), “Total number of lessons”, or “Total course duration” change continuously when content is updated. If every time users view the course list, the system must query the entire Comment, Lesson, and Module tables to recalculate (SUM, AVG), database performance will seriously degrade (N+1 Query issue and Aggregation cost). Conversely, if stored permanently in the database but not updated in time, the data will be inaccurate.

5.4.2 Proposed Solution

The student applied an Event-Driven Updates model combined with Aggregate Fields to solve performance and consistency issues.

Detailed data structure has been described in Table 4.2.2.2 in Chapter 4.

Specific solution:

- **Using Transient fields and caching:** In the object-oriented model, fields such as `rating` and `lessonsCount` are defined as Transient (not stored directly but computed). However, for optimization, the student applied caching at the Service layer.

- **Trigger/Callback mechanism:**

- When a new Comment is approved, the system triggers an event to update the average Rating and stores it in cache or a separate statistics table.
- When a Lesson is added/removed, the `totalDuration` field of the course is updated incrementally (Incremental Update) instead of recalculating from scratch.

- **computeStatistics method:** Optimized to process bulk data in memory (In-memory processing) before returning to the client, minimizing the number of database queries.

5.4.3 Achieved Results

- The course listing page loading time decreased from 1.5s to under 200ms by eliminating complex aggregation queries.
- Displayed data always ensures “Real-time” updates immediately after changes occur.

5.5 Secure Payment Integration with VNPay and Digital Signature

5.5.1 Problem Statement

Online payment integration is a mandatory requirement for e-commerce/education systems. The biggest issue is transaction security. Attackers may modify the amount in transit (Man-in-the-middle attack) or forge requests (Replay attack) to enroll in courses without paying. In addition, the system must handle cases where users finish payment but immediately close the browser (not waiting for redirect).

5.5.2 Proposed Solution

The student built a payment module integrating the VNPay gateway in compliance with the highest security standards.

The solution includes:

- **Data integrity authentication mechanism (Data Integrity):** Using the HMAC-SHA512 algorithm to generate a digital signature (Checksum) for all outgoing and incoming data.
 - When generating the payment URL: $\text{Signature} = \text{HMAC_SHA512}(\text{SecretKey}, \text{DataString})$.
 - When receiving the Callback: The system recalculates the signature from received data and compares it with the signature included by VNPay. If it differs by even 1 character, the transaction is rejected.

- **IPN (Instant Payment Notification):** Not relying only on browser redirect, the system builds a Listening API to receive hidden transaction status notifications from the VNPay server. This ensures the course is activated even if the user loses network connection after payment.
- **Transaction Logging:** Logging the entire transaction status flow (PENDING → PAID/FAILED) into the Payment table for reconciliation and dispute handling.

5.5.3 Achieved Results

- The payment system operates stably and accurately. No incorrect order-status cases were recorded during testing.
- The process “Successful payment → Automatic course activation → Confirmation email sending” is fully automated without manual intervention.

5.6 Standard RESTful API Architecture and Multi-layer Security

5.6.1 Problem Statement

A system serving multiple platforms (Web, future Mobile) needs a standardized Application Programming Interface (API). At the same time, API security is critical to prevent user data leakage (IDOR, Injection).

5.6.2 Proposed Solution

The student designed the backend system following a high-completeness RESTful API standard (Richardson Maturity Model Level 2), combined with Multi-layer Security.

Specific solution:

- **Standardized Response:** All APIs return data in a unified JSON structure (including `status`, `message`, `data`, `timestamp`), helping the Frontend handle errors easily (Error Handling).
- **Stateless authentication with JWT:** Using JSON Web Token for authentication, so the system does not need to store sessions on the server and can scale out easily.
- **3-layer security:**
 - Layer 1 (Network): CORS Filter allows only the Frontend domain to access.
 - Layer 2 (Authentication): `JwtAuthenticationFilter` verifies user identity.
 - Layer 3 (Authorization): Method Security (`@PreAuthorize`) checks

specific permissions on each handler method.

5.6.3 Achieved Results

- The API system is clear and easy to understand, facilitating future Mobile App development.
- Ensures data safety and prevents unauthorized access to other users' resources.

5.7 Summary

Chapter 5 presented in detail the 06 core solutions and contributions of the thesis. From solving the difficult technical problem of secure code execution (Judge0 + Docker), applying advanced AI technology (Gemini) to enhance the learning experience, to building rigorous business workflows for Teaching Assistants (RBAC) and Payment (VNPay).

These solutions are not only theoretical but have been successfully implemented in the product, demonstrating the correctness of the design and the high practical applicability of the system. This is a solid foundation for continuing development and expansion of the system in the future.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

Throughout the research and implementation of the graduation thesis, the student conducted surveys and compared the product with popular online education platforms today such as Udemy, Coursera, and CodeLearn. The comparison results show that although the system built within the scope of the thesis cannot yet possess as rich a content repository as large commercial platforms, it demonstrates superior advantages in specialization for programming teaching. The highlight lies in the ability to integrate a programming environment (IDE) and an automatic grading system directly in the browser, along with high interactivity through an AI virtual assistant and a real-time chat system. Furthermore, the business process for the Teaching Assistant role is designed closely to the practical teaching context at universities in Vietnam, a feature that international platforms often do not fully satisfy.

In summary of the implementation process, the student has completed the goal of building a complete Learning Management System (LMS) that meets the needs of online programming teaching and learning well. In terms of functionality, the system has completed a closed business workflow from registration, payment, learning participation, exercise practice, automatic grading, to issuing completion certificates. In terms of technical aspects, the thesis successfully solved difficult problems such as integrating the Judge0 API to grade source code securely in a Docker Sandboxing environment, as well as applying Google Gemini AI to intelligently support learners in debugging source code. In addition, system performance is significantly optimized thanks to applying caching techniques, asynchronous processing, and a decoupled service architecture, helping the system operate stably and respond quickly. Security and safety factors are also ensured through implementing JWT authentication mechanisms, multi-layer API security, and integrating the VNPay payment gateway with secure digital signatures.

However, besides the achieved results, the thesis still has certain limitations. The mobile interface, although supporting Responsive Design, still does not provide a truly convenient source code writing experience due to limitations of display space and the virtual keyboard. The system load capacity has also not been verified through stress tests at the scale of thousands of concurrent users to evaluate the real limits of the infrastructure. In addition, the current AI virtual assistant feature only stops at the level of explaining errors and suggesting fixes, and has not yet been

able to automatically build personalized learning pathways based on the history and behavior of each learner.

Through the thesis implementation process, the student has drawn many valuable lessons. First is system design thinking, helping the student understand the importance of building a solid architecture before starting to code, especially separating services for easier future scaling. Next is project management skills, including breaking down large problems into specific tasks, managing time, and controlling source code versions effectively. Finally, the student's problem-solving ability was also significantly improved through self-studying new technologies such as Docker, AI Integration, and handling issues arising during third-party system integration.

6.2 Future Work

To refine the product and enhance practical applicability in the future, the student proposes development directions focusing on improving existing functions and expanding new research directions.

First, for the current functions, the system needs to upgrade the source code editor (Code Editor) by integrating more intelligent code suggestion features (IntelliSense) and stronger automatic code formatting, similar to the experience on VS Code, in order to increase convenience for learners. In addition, expanding the test case set (Test Case) is also an urgent requirement, through building a tool to support instructors in creating inputs/outputs more visually, while also supporting unit testing (Unit Test) for popular languages such as Java or Python. In terms of deployment, a complete continuous integration and continuous deployment (CI/CD) process needs to be established to automate testing and deploy the application to a cloud computing environment, replacing the current manual deployment method.

Regarding new directions, developing a separate native mobile application (Native Mobile App) using React Native or Flutter will be an important step to fully leverage device features such as push notifications or offline learning modes. At the same time, applying artificial intelligence and machine learning (Machine Learning) to analyze learning behavior data will open up the ability to personalize learning pathways, helping recommend the most suitable courses and exercises according to each learner's level and interests. In addition, a coding collaboration feature (Coding Collaboration) that allows instructors and learners to work together on one screen in real time is also a potential development direction, serving effectively for direct guidance activities. Finally, when the user scale increases, transitioning from a monolithic architecture (Monolithic) to a fully microservices architecture

(Microservices) will be the optimal solution to increase independent scalability for each functional subsystem such as user management, courses, grading, or payment.

REFERENCE

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [2] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022.
- [3] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [4] B. Cheang, A. Kurnia, A. Lim, and W. Oon, "On automated grading of programming assignments in an academic institution," *Computers & Education*, vol. 41, no. 2, pp. 121–131, 2003.
- [5] D. Crockford, *JavaScript: The Good Parts*. O'Reilly Media, 2008.
- [6] Oracle Corporation, *Java platform, standard edition documentation*, [Online]. Available: <https://docs.oracle.com/en/java/javase/index.html> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/index.html>.
- [7] Meta Platforms, *React - a javascript library for building user interfaces*, [Online]. Available: <https://react.dev/reference/react> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://react.dev/reference/react>.
- [8] VMware, Inc., *Spring boot documentation*, [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>.
- [9] Judge0, *Judge0 api documentation*, [Online]. Available: <https://ce.judge0.com/> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://ce.judge0.com/>.
- [10] Docker Inc., *Docker documentation*, [Online]. Available: <https://docs.docker.com/> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://docs.docker.com/>.
- [11] Google, *Gemini api overview*, [Online]. Available: <https://ai.google.dev/docs> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://ai.google.dev/docs>.
- [12] M. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," IETF, RFC 7519, 2015, [Online]. Available: <https://tools.ietf.org/html/rfc7519> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://tools.ietf.org/html/rfc7519>.

- [13] VNPAY, *Tài liệu tích hợp cổng thanh toán vnpay-qr*, [Online]. Available: <https://sandbox.vnpayment.vn/apis/docs/huong-dan-tich-hop/> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://sandbox.vnpayment.vn/apis/docs/huong-dan-tich-hop/>.
- [14] MySQL AB, *Mysql 8.0 reference manual*, [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/> (visited on 01/15/2026). Accessed: Jan. 15, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>.

APPENDIX